P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

Department of Information Technology

FINAL PROJECT REPORT

(Hard-Bound Academic Project Report)

"Cache-Craft: Autonomous Semantic RAG Router"

A High-Performance Multi-Tier Semantic Caching Middleware for LLM Inference

A Project Report Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Technology in Information Technology
(7th Semester)

Candidate Name	**AAKASH BHARATBHAI JAYANI**
Enrollment Number	**23SE02IT140**
Branch / Discipline	B.Tech Information Technology
Project Classification	UDP (User Defined Project)
Institute Guide / Mentor	**Mr. Anurag Yadav**
Academic Year	2026 - 2027

Surat, Gujarat, India — OCTOBER 2026

P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

CERTIFICATE OF COMPLETION

This is to certify that Mr. AAKASH BHARATBHAI JAYANI, Enrollment No.: 23SE02IT140, a bonafide student of B.Tech Information Technology, 7th Semester, has successfully completed the User Defined Project (UDP) titled "Cache-Craft: Autonomous Semantic RAG Router - A High-Performance Multi-Tier Semantic Caching Middleware for LLM Inference" during the academic period from 08/06/2026 to 03/10/2026 at P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, Department of Information Technology.

The project work, algorithmic implementation, multi-tier caching engine, empirical benchmark evaluation on the 1,200-query corpus, full-stack Next.js web application, and comprehensive documentation embodied in this report have been thoroughly investigated, implemented, and validated under our continuous supervision. The work demonstrated satisfies the academic and engineering standards prescribed by the University for the degree of Bachelor of Technology in Information Technology.

Date of Issue: 03 / 10 / 2026 **Place: Surat, Gujarat, India**

_____	_____
Mr. Anurag Yadav Institute Project Guide / Mentor Department of Information Technology	Head of Department Department of Information Technology School of Engineering, PPSU
_____	_____
External Examiner Signature	Dean / Principal School of Engineering, PPSU

CANDIDATE DECLARATION

I, AAKASH BHARATBHAI JAYANI, Enrollment Number 23SE02IT140, hereby declare that this final project report entitled "Cache-Craft: Autonomous Semantic RAG Router - A High-Performance Multi-Tier Semantic Caching Middleware for LLM Inference" is a consolidated, authentic, and original record of the engineering research and development work carried out by me as a User Defined Project (UDP) during the 7th Semester of the B.Tech program in Information Technology at P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, Surat.

I further declare that this report represents my own intellectual and engineering efforts under the supervision of my Institute Mentor, Mr. Anurag Yadav. The codebase, empirical benchmark data across 1,200 corpus queries, system architecture diagrams, user interface components, and performance measurements presented herein are genuine and have not been submitted in part or full for the award of any other degree, diploma, fellowship, or similar title to any other university or institute.

All literature references, open-source software libraries, datasets, and foundational models (including PostgreSQL pgvector, sentence-transformers, Redis, Google Gemini, Ollama llama3.2, and Next.js) utilized during this work have been duly acknowledged and cited in accordance with academic standards and APA 7th edition formatting.

_____

AAKASH BHARATBHAI JAYANI
Enrollment No.: 23SE02IT140
B.Tech Information Technology, 7th Semester
P P Savani University, Surat

Date: 03 / 10 / 2026 Place: Surat, Gujarat, India

ACKNOWLEDGEMENT

I take immense pleasure in expressing my profound gratitude and indebtedness to P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, and the Department of Information Technology for providing an intellectually stimulating environment and outstanding computational facilities that made the successful execution of this User Defined Project possible.

I express my deepest gratitude to my esteemed Institute Guide and Mentor, Mr. Anurag Yadav, for his invaluable guidance, keen technical insights, constant encouragement, and constructive critique throughout the developmental lifecycle of Cache-Craft. His deep knowledge of distributed systems and machine learning pipelines proved instrumental in shaping the multi-tier routing architecture, resolving complex vector distance space anomalies, and establishing rigorous empirical benchmarking methodologies.

I also extend my sincere appreciation to the Head of the Department and the faculty members of the Department of Information Technology for their continuous academic support, constructive evaluations during the four milestone reporting defenses, and helpful feedback on system design and validation.

Special thanks are due to my peers and fellow researchers who participated in user acceptance testing and provided insightful feedback on the interactive chat playground and real-time observability telemetry. Lastly, I am eternally grateful to my parents and family for their unwavering moral support, patience, and encouragement throughout my academic journey.

AAKASH BHARATBHAI JAYANI

Enrollment No.: 23SE02IT140

ABSTRACT / EXECUTIVE SUMMARY

The rapid industrial proliferation of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) pipelines has created a critical operational bottleneck characterized by excessive inference latency (often exceeding 2,000 to 4,000 milliseconds) and steep recurring financial expenditures driven by per-token cloud API billing. While traditional web architectures rely heavily on deterministic key-value caching, conventional exact-match caches fail entirely in generative AI workflows because natural language queries expressing identical semantic intent exhibit infinite syntactic variations. To overcome this fundamental limitation, this project designs, implements, and empirically evaluates Cache-Craft: an autonomous, multi-tier semantic RAG router and intelligent caching middleware.

Cache-Craft introduces an engineered three-tier hierarchical architecture consisting of: (1) an L0 sub-millisecond SHA-256 exact match bypass, (2) an L1 in-memory LRU session buffer, and (3) an L2 semantic vector cache powered by sentence-transformers/all-MiniLM-L6-v2 (384-dimensional dense embeddings) coupled with a PostgreSQL 16 pgvector Hierarchical Navigable Small World (HNSW) graph index. Incoming natural language queries are projected into a unit-normalized dense vector space, and semantic similarity is evaluated using an optimized cosine distance metric against a dynamically configurable similarity threshold (tau). When semantic similarity meets or exceeds the threshold, verified responses are delivered in approximately 25 milliseconds, completely bypassing costly LLM generation. When a semantic miss occurs, queries are routed to a dual-provider inference engine featuring Google Gemini 3.8 Flash with an automated, transparent fallback to an offline local Ollama llama3.2 daemon during HTTP 503 errors or quota limits.

Rigorous empirical validation conducted across a curated 1,200-query benchmark corpus (500 pre-seeded queries, 500 natural language paraphrases, and 200 out-of-domain negative controls) demonstrated that an optimal threshold of tau = 0.70 achieves an F1-score of 0.9806 with 100.0% precision (zero false positives) and 96.20% semantic recall. Under realistic evaluation workloads, Cache-Craft achieved a 68.71% semantic hit rate, reducing average end-to-end query latency from 2,400.0 ms to 768.26 ms (a 3.2x overall speedup) while decreasing token API expenditures by exactly 68.71%. The entire system is packaged with an enterprise-grade Next.js

15 dark-glassmorphism dashboard featuring real-time Server-Sent Events (SSE) telemetry, interactive prompt history navigation, and hardware device authorization boundaries.

Keywords: *Semantic Caching, Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), pgvector, Hierarchical Navigable Small World (HNSW), Cosine Similarity, FastAPI, Next.js 15, Inference Acceleration, Token Cost Reduction, Ollama, Google Gemini.*

TABLE OF CONTENTS

Chapter / Section	Title / Content Description	Page No.
Preliminary	Certificate of Completion	i
Preliminary	Candidate Declaration	ii
Preliminary	Acknowledgement	iii
Preliminary	Abstract / Executive Summary	iv
Preliminary	Table of Contents	vi
Preliminary	List of Figures	xi
Preliminary	List of Tables	xii
Preliminary	List of Abbreviations and Acronyms	xiv
CHAPTER 1	INTRODUCTION	1
1.1	Background of the Project	1
1.2	Problem Statement	2
1.3	Need / Motivation for the Project	3
1.4	Objectives of the Project	3
1.5	Scope of the Project & Scope Matrix (Table 1.1)	4
1.6	Expected Outcomes vs. Final Achieved Outcomes (Table 1.2)	5
1.7	Organization of the Report	6
CHAPTER 2	ORGANIZATION / UDP PROFILE	7
2.1	Organization / Institutional Overview (P P Savani University)	7
2.2	Department / Domain Profile (IT & AI Systems)	7
2.3	Role and Individual Responsibilities	7
2.4	Development / Work Environment & Specifications (Table 2.1)	8
2.5	UDP Project Administrative Details	9
CHAPTER 3	LITERATURE REVIEW / BACKGROUND STUDY	10
3.1	Domain Background: Semantic Search & Inference Economics	10
3.2	Existing Approaches & Related Work (Table 3.1)	10

Chapter / Section	Title / Content Description	Page No.
3.3	Comparative Analysis (Table 3.2)	11
3.4	Research Gap and Project Justification	12
CHAPTER 4	REQUIREMENTS ANALYSIS AND PROJECT SCOPE	14
4.1	Functional Requirements Specification (Table 4.1)	14
4.2	Non-Functional Requirements Specification (Table 4.2)	15
4.3	Hardware Requirements and Infrastructure (Table 4.3)	16
4.4	Software Requirements and Dependencies (Table 4.4)	16
4.5	Data / Dataset Requirements (Table 4.5)	17
4.6	System Constraints and Operational Assumptions	18
CHAPTER 5	METHODOLOGY AND PROJECT PLANNING	19
5.1	Final Development and Research Methodology	19
5.2	Project Workflow & Process Architecture (Figure 5.1)	19
5.3	Project Phases and Deliverables (Table 5.1)	20
5.4	Progress Report Consolidation (Reports 1 to 4) (Table 5.2)	21
5.5	Project Schedule and 16-Week Gantt Timeline (Table 5.3)	21
CHAPTER 6	SYSTEM ANALYSIS AND DESIGN	23
6.1	Existing System Analysis & Bottleneck Identification	23
6.2	Proposed Multi-Tier Autonomous System Architecture	23
6.3	High-Performance System Architecture (Figure 6.1)	24
6.4	Autonomous Routing Decision Tree & Workflow (Figure 6.2)	25
6.5	UML Class, Sequence, Component & Block Diagrams (Figures 6.3 & 6.4)	25
6.6	Module-wise Detailed Design (Table 6.1)	27
6.7	Database / Relational pgvector Data Design (Table 6.2)	29
6.8	User Interface & Experience Design System	30
CHAPTER 7	TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT	32
7.1	Programming Languages (Python 3.11, TypeScript, SQL)	32

Chapter / Section	Title / Content Description	Page No.
7.2	Frameworks, Libraries & External APIs	32
7.3	Database & In-Memory Storage Systems	32
7.4	Hardware, Cloud & Deployment Environments	32
7.5	Development, Version Control & Testing Tooling	33
7.6	Technology Selection Rationale & Trade-Off Analysis (Table 7.6)	33
CHAPTER 8	SYSTEM IMPLEMENTATION	35
8.1	Implementation Architecture Overview	35
8.2	Module 1: Multi-Tier Cache Router Core	35
8.3	Module 2: Dense Vector Embedding & Normalization Engine	35
8.4	Module 3: Dynamic Multi-LLM Router & Failover Engine	35
8.5	Module 4: Hardware Device Authorization & Isolation Boundary	35
8.6	Module 5: Real-Time Observability & SSE Telemetry Engine	36
8.7	Module 6: Interactive Web Dashboard & Chat Playground	36
8.8	Module 7: Automated Benchmarking & Matrix Validation Suite	36
8.9	Mathematical Formulations: Cosine Metric & HNSW Graph Indexing	36
8.10	RESTful API Endpoints Specification (Table 8.1)	37
8.11	Database Implementation & pgvector DDL Excerpts	38
8.12	Security Architecture, CORS & GPU Access Control	39
8.13	End-to-End System Integration	39
CHAPTER 9	TESTING AND VALIDATION	40
9.1	Multi-Tiered Testing Strategy & Verification Methodology	40
9.2	Unit and Module Testing Suite (25 Tests) (Table 9.1)	40
9.3	Functional Testing Suite (40 Tests) (Table 9.2)	41
9.4	Integration Testing Suite (35 Tests) (Table 9.3)	42
9.5	System and Resilience Testing Suite (25 Tests) (Table 9.4)	43
9.6	Performance, Load & Concurrency Testing Suite (20 Tests) (Table 9.5)	44

Chapter / Section	Title / Content Description	Page No.
9.7	User Acceptance Testing (UAT) Scenarios (18 Tests) (Table 9.6)	45
9.8	Experimental Validation on 1,200 Benchmark Corpus	46
9.9	Defects Discovered, Root Causes & Resolutions (Table 9.7)	46
9.10	Formal Validation Statement (163/163 Tests Passed)	48
CHAPTER 10	RESULTS AND PERFORMANCE ANALYSIS	49
10.1	Final Empirical Results Summary	49
10.2	Objective-wise Milestone Achievement (Table 10.1)	49
10.3	Similarity Threshold Sweep & Metric Analysis (Table 10.2)	50
10.4	Baseline Comparative Results (Table 10.3)	50
10.5	Empirical Charts & Production Screenshots (Figures 10.1 to 10.6)	51
10.6	Results Discussion & Economic Cost Savings Analysis	54
CHAPTER 11	CHALLENGES, SOLUTIONS AND IMPROVEMENTS	55
11.1	Major Architectural & Algorithmic Challenges	55
11.2	Technical Challenges & Engineering Resolutions	55
11.3	Process, Dataset & Environment Challenges	56
11.4	Architectural Improvements Made During Development (Table 11.1)	56
11.5	Lessons Learned from Systems Engineering Challenges	57
CHAPTER 12	INDIVIDUAL CONTRIBUTION AND PROFESSIONAL LEARNING	58
12.1	Individual Contribution & Project Ownership (100% Solo Ownership)	58
12.2	Technical Competencies & Skills Acquired (Table 12.1)	58
12.3	Professional, Research & Architectural Skills Acquired (Table 12.2)	59
12.4	Major Academic and Professional Learning Outcomes	59
CHAPTER 13	LIMITATIONS AND FUTURE SCOPE	60
13.1	Current Technical Limitations (Table 13.1)	60
13.2	Future Scope & Developmental Roadmap (Table 13.2)	60

Chapter / Section	Title / Content Description	Page No.
13.3	Scalability, Clustering & Enterprise Extension Opportunities	61
CHAPTER 14	CONCLUSION	62
14.1	Final Project Completion Status (Table 14.1)	62
14.2	Concluding Remarks & Architectural Significance	62
CHAPTER 15	REFERENCES (APA 7th Edition)	64
APPENDICES	SUPPORTING DOCUMENTS AND EVIDENCE	66
Appendix A	Progress Report 1 Summary (Foundations & Concept)	66
Appendix B	Progress Report 2 Summary (Prototype & Proof)	66
Appendix C	Progress Report 3 Summary (pgvector & Frontend V1)	66
Appendix D	Reporting 4 / Final Evaluation Summary	67
Appendix E	Additional Benchmark Visualizations & Data Callouts	67
Appendix F	Certificates & Academic Evidence (NPTEL OS Certificate)	67
Appendix G	Source Code, Git Repository & Cloud Deployment Evidence	68
Appendix H	Mentor Evaluation & Continuous Review Rubrics	69
ANNEXURE	Final Hard-Bound Submission Checklist (Table Annexure.1)	71
GUIDELINES	PPSU School of Engineering Formatting Compliance Statement	73

LIST OF FIGURES

Figure No.	Figure Caption / Architectural Description	Page No.
Figure 5.1	Cache-Craft End-to-End System Development and Research Workflow	20
Figure 6.1	High-Performance Multi-Tier System Architecture Diagram (L0/L1/L2)	24
Figure 6.2	Autonomous Semantic Routing Decision Tree and Multi-Tier Logic Flowchart	25
Figure 6.3	Multi-Panel UML Class, Sequence, and Component Architecture Diagrams	26
Figure 6.4	Hardware Infrastructure and Middleware Layer Physical Block Diagram	27
Figure 10.1	Empirical Latency Profiling and Cloud Cost Reduction Benchmark Curves	51
Figure 10.2	Threshold Sensitivity vs. Precision, Recall, and F1-Score Trade-Off Analysis	52
Figure 10.3	Next.js 15 Web Dashboard and Real-Time Telemetry Overview	52
Figure 10.4	Interactive Semantic Chat Playground with L1 Cache-Hit Highlighting	53
Figure 10.5	Vector Cache Explorer and Semantic Inverted Index Browser	53
Figure 10.6	Dynamic Similarity Threshold Controls and Device Authorization Panel	54
Figure F.1	NPTEL Operating Systems Fundamentals and Multithreading Certification	68

LIST OF TABLES

Table No.	Table Title / Context	Page No.
Table 1.1	Cache-Craft Project Scope Matrix (In Scope vs. Out of Scope)	4
Table 1.2	Expected Outcomes vs. Final Achieved Outcomes	5
Table 2.1	UDP Workstation Hardware and System Specifications	8
Table 3.1	Overview of Existing Semantic Caching and RAG Routing Approaches	10
Table 3.2	Comprehensive Feature-by-Feature Comparative Matrix	11
Table 4.1	Functional Requirements Specification (FR-01 to FR-10)	14
Table 4.2	Non-Functional Requirements Specification (NFR-01 to NFR-08)	15
Table 4.3	Hardware Requirements and Infrastructure Specifications	16
Table 4.4	Software Requirements, Frameworks, and Runtime Dependencies	16
Table 4.5	Benchmark Dataset Requirements and Corpus Distribution (1,200 Queries)	18
Table 5.1	Project Phases, Milestones, and Key Deliverables	20
Table 5.2	Progress Report Consolidation and Developmental Evolution (Reports 1 to 4)	21
Table 5.3	Comprehensive 16-Week Project Schedule and Timeline (Gantt Representation)	21
Table 6.1	Module-wise Architectural Design and Functional Responsibilities	27
Table 6.2	PostgreSQL Database Schema and pgvector Index Definitions	29
Table 7.6	Technology Selection Rationale and Architectural Trade-Off Analysis	33
Table 8.1	Cache-Craft RESTful API Endpoints Specification	37
Table 9.1	Unit and Module Testing Suite Test Cases (25 Tests)	40
Table 9.2	Functional Testing Suite Test Cases (40 Tests)	41
Table 9.3	Integration Testing Suite Test Cases (35 Tests)	42
Table 9.4	End-to-End System and Resilience Testing Test Cases (25 Tests)	43
Table 9.5	Performance, Load, and Concurrency Testing Test Cases (20 Tests)	44
Table 9.6	User Acceptance Testing (UAT) Usability Scenarios (18 Tests)	45

Table No.	Table Title / Context	Page No.
Table 9.7	Major Defects Discovered, Root Causes, and Technical Resolutions	46
Table 10.1	Objective-wise Milestone and Metric Achievement Summary	49
Table 10.2	Empirical Similarity Threshold Sweep and Benchmark Evaluation (N=700)	50
Table 10.3	Baseline Comparative Results (Direct LLM vs. Exact Hash vs. Cache-Craft)	51
Table 11.1	Architectural Improvements and Developmental Refinements	56
Table 12.1	Technical Skills and Engineering Competencies Acquired	58
Table 12.2	Professional, Research, and Project Management Skills Acquired	59
Table 13.1	Current System Limitations and Operational Boundaries	60
Table 13.2	Future Scope, Planned Enhancements, and Scalability Opportunities	61
Table 14.1	Final Comprehensive Project Completion Status Matrix	62
Table H.1	Final Project Evaluation Rubric and Grading Criteria	69
Table Annexure.1	Final Hard-Bound Submission Checklist (17 Verification Points)	71

LIST OF ABBREVIATIONS / ACRONYMS

Abbreviation / Acronym	Full Expansion and Engineering Context
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
ANN	Approximate Nearest Neighbor
HNSW	Hierarchical Navigable Small World (Graph Index)
SSE	Server-Sent Events
TTL	Time-To-Live (Cache Expiration Lifecycle)
SHA	Secure Hash Algorithm (specifically SHA-256)
COS	Cosine Similarity / Cosine Distance Metric
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
DDL	Data Definition Language
SQL	Structured Query Language
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
UAT	User Acceptance Testing
FR	Functional Requirement
NFR	Non-Functional Requirement
UI	User Interface
UX	User Experience
CI/CD	Continuous Integration and Continuous Deployment
MVC	Model-View-Controller Architectural Pattern
HTTP	Hypertext Transfer Protocol

Abbreviation / Acronym	Full Expansion and Engineering Context
HTTPS	Hypertext Transfer Protocol Secure
TCP	Transmission Control Protocol
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture (NVIDIA)
IVF	Inverted File Index (Vector Quantization)
SLA	Service Level Agreement
UDP	User Defined Project (Academic Track)

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

The contemporary computing landscape is experiencing an epochal paradigm shift driven by the rapid maturation and industrial integration of Large Language Models (LLMs) such as OpenAI's GPT-4, Google's Gemini, and Meta's Llama series. Enterprises and academic institutions increasingly deploy these models within Retrieval-Augmented Generation (RAG) pipelines to deliver conversational intelligence, code synthesis, document summarization, and customer support. By anchoring foundation models to external vector knowledge repositories, RAG frameworks dramatically reduce factual hallucinations and empower systems with domain-specific precision.

However, this widespread adoption has exposed severe operational vulnerabilities in modern AI architectures. Foundation model inference is compute-intensive, memory-bound, and economically costly. Cloud providers bill LLM access on a granular per-token consumption model, transforming high-traffic conversational workloads into substantial, recurring operational expenses. Concurrently, inference latency frequently ranges from 1,500 to over 4,000 milliseconds per query, severely degrading the user experience in interactive and real-time environments.

In traditional web systems, high latency and repeated computational costs are systematically mitigated through deterministic key-value caching mechanisms (such as Memcached or standard Redis). Unfortunately, deterministic caching relies on exact cryptographic key matching (e.g., SHA-256 string hashes). In generative AI applications, natural language queries expressing identical underlying user intent exhibit infinite lexical and syntactic variations (e.g., 'What is a semantic cache?' versus 'Can you explain semantic caching?'). Exact-match caches experience a near 100% miss rate against paraphrased inputs, forcing the underlying system to re-invoke expensive LLM inference repeatedly. Cache-Craft was conceived to solve this fundamental architectural bottleneck through high-performance semantic routing and multi-tier vector caching.

1.2 Problem Statement

Modern LLM-powered applications suffer from three interrelated systemic deficiencies that constrain their scalability:

- **1. Redundant Cloud API Expenditures:** In conversational RAG systems, up to 60–70% of inbound queries in customer support and documentation search express semantic intent identical to previously resolved prompts. Due to lexical variations, current systems re-execute the entire RAG pipeline, generating millions of redundant tokens and wasting financial capital.
- **2. Unacceptable Inference Latency:** End-to-end generation times spanning 2 to 4 seconds introduce cognitive friction, degrade conversational responsiveness, and prevent integration into latency-critical enterprise SLAs.
- **3. Single-Point-of-Failure Fragility:** Cloud LLM APIs are inherently susceptible to transient network congestion, rate limiting (HTTP 429), and upstream service outages (HTTP 503). Relying exclusively on single-provider cloud endpoints introduces a brittle single point of failure that halts customer workflows.

Formally, the engineering problem addressed by this project is: How can we architect an autonomous, low-overhead middleware that accurately determines semantic equivalence between incoming natural language prompts and historical responses in real time (<30 ms), serves verified answers from an optimized multi-tier cache, and dynamically orchestrates fallback between cloud and local LLM providers without sacrificing contextual precision or security?

1.3 Need / Motivation for the Project

The motivation for Cache-Craft stems from both academic necessity and pressing industrial demand. As generative AI transitions from experimental prototypes to mission-critical infrastructure, computational efficiency and operational autonomy become paramount. Organizations cannot sustain exponentially growing cloud bills for queries their systems have already answered thousands of times. Furthermore, data privacy mandates require mechanisms that can keep sensitive user queries confined to local hardware without sacrificing performance.

By constructing a multi-tier semantic router that combines sub-millisecond cryptographic hash checks with dense vector similarity searches over an ACID-compliant PostgreSQL 16 pgvector store, Cache-Craft provides an enterprise-ready bridge between traditional caching performance and modern generative AI flexibility. Crucially, providing an automated local fallback engine (via Ollama

llama3.2) alongside cloud intelligence (Google Gemini) democratizes resilient AI infrastructure.

1.4 Objectives of the Project

The primary engineering and research objectives established for Cache-Craft are:

- **1. Multi-Tier Cache Hierarchy:**Construct a three-tier caching hierarchy incorporating an L0 sub-millisecond SHA-256 exact match bypass, an L1 in-memory session buffer, and an L2 persistent vector semantic cache.
- **2. Real-Time Vector Embedding:**Integrate the lightweight sentence-transformers/all-MiniLM-L6-v2 model to project natural language queries into 384-dimensional unit-normalized dense embeddings within 10 milliseconds.
- **3. Sub-30ms Semantic Similarity Search:**Implement an accelerated Hierarchical Navigable Small World (HNSW) graph index in PostgreSQL pgvector to evaluate cosine distance (<=>) in sub-30 milliseconds.
- **4. Rigorous Mathematical Optimization:**Empirically evaluate precision, recall, F1-score, and cost curves across a curated 1,200-query benchmark corpus, identifying the optimal mathematical threshold (tau) to achieve 100% precision.
- **5. Fault-Tolerant Multi-LLM Routing:**Architect an automated fallback pipeline that seamlessly redirects traffic from Google Gemini Cloud to a local Ollama llama3.2 instance during API outages or rate limit spikes.
- **6. Multi-Turn Context-Aware Caching:**Implement referential intent detection and partition filtering in pgvector to prevent context collision and semantic drift between root queries and conversational follow-ups.
- **7. Modern Full-Stack Web Platform:**Build an Apple-grade, responsive Next.js 15 dashboard featuring real-time Server-Sent Events (SSE) telemetry, chat playground, prompt history recall, and hardware device security boundaries.

1.5 Scope of the Project

The architectural scope of Cache-Craft covers the complete lifecycle of incoming natural language queries within text-based RAG and LLM systems. The project encompasses embedding generation, multi-tier cache evaluation, vector database indexing, cloud and local LLM orchestration, empirical benchmarking, and web telemetry. To maintain rigorous software engineering boundaries, Table 1.1 delineates the explicit scope matrix.

Table 1.1: Cache-Craft Project Scope Matrix

In Scope (Delivered & Validated)	Out of Scope (Boundaries)	Architectural Remarks
Multi-Tier Cache Routing (L0 SHA-256 + L1 Memory + L2 pgvector)	Cross-modal image/audio embedding caching	L0 provides sub-millisecond hash hits; L2 semantic cache handles dense text vectors.
Pluggable Dual-LLM Inference (Ollama llama3.2 + Google Gemini API)	Training or fine-tuning underlying foundation LLM weights	Focus is on high-performance inference routing, token reduction, and cloud-to-local fallback.
Hardware Device Authorization & GPU Boundary Protection	Public user login/registration with OAuth2 credentials	Secures local GPU resources from external client exploitation while allowing open API access.
Automated Cloud-to-Local Fallback on HTTP 503/Rate Limits	Distributed multi-region active-active cluster failover	Ensures 100% inference availability on standalone deployment environments.
Real-Time Streaming Telemetry (SSE) & Interactive Next.js 15 UI	Third-party payment gateways or billing subscription engines	Provides live latency breakdown, hit rate gauges, and interactive chat playground.
Empirical Multi-Threshold Benchmarking on 1,200 Query Corpus	Benchmarking proprietary closed models beyond Gemini Flash	Rigorous evaluation validates precision, recall, and cost curves across similarity thresholds.

1.6 Expected Outcomes vs. Final Achieved Outcomes

Throughout the four reporting milestones of the UDP lifecycle, project deliverables were tracked against quantitative engineering benchmarks. Table 1.2 presents the comparative evaluation of initial expectations versus final verified outcomes.

Table 1.2: Expected vs. Final Achieved Project Outcomes

Performance Parameter	Initial Expected Target	Final Achieved Outcome (Empirical)
Semantic Similarity Hit Rate	Achieve ≥60% semantic cache hit rate on paraphrased natural language queries.	Achieved 68.71% hit rate on evaluation corpus ($N=700$) at optimal similarity threshold $\tau=0.70$.
Cache Hit Latency Reduction	Serve cached queries in under 50 ms compared to >2,000 ms direct LLM inference.	Achieved 25.33 ms average L2 semantic hit latency and 1.84 ms L1 exact match latency (up to 99% reduction).

Performance Parameter	Initial Expected Target	Final Achieved Outcome (Empirical)
Inference Latency Speedup	Achieve overall end-to-end speedup factor ≥2.5× across mixed query workloads.	Achieved 3.2× overall speedup factor (reducing average latency from 2,400 ms to 768.26 ms across corpus).
Cloud API Cost Reduction	Reduce external LLM token consumption and API billing costs by ≥50%.	Achieved 68.71% token cost reduction, cutting expenditure from $0.2800 to $0.0876 on evaluation set.
Semantic Precision SLA	Maintain semantic precision ≥95% to prevent false positives and context hallucinations.	Achieved 100.0% precision (0 false positives) at $\tau=0.70$ with F1-score of 0.9806.
Fault Tolerant Fallback	Gracefully handle external cloud API outages (503/429) without crashing client sessions.	Engineered zero-downtime automated fallback from Google Gemini to local Ollama llama3.2 daemon.
Multi-Turn Context Caching	Prevent semantic drift and context collision when caching follow-up questions.	Implemented referential intent detection and pgvector partition isolation for follow-up query turns.
Production Web Dashboard	Deliver modern interactive UI for real-time monitoring and parameter tuning.	Built Next.js 15 glassmorphism dashboard with Overview, Chat, Cache Explorer, and Settings panels.

1.7 Report Organization

This final hard-bound report is structured into fifteen cohesive chapters and supporting appendices, following the official P P Savani University School of Engineering formatting standards:

- **Chapter 2 (Organization / UDP Profile):**Details institutional background, IT department domain, individual role, and development environment.
- **Chapter 3 (Literature Review):**Surveys existing caching literature (GPTCache, RedisVL, Towhee) and establishes the research gap.
- **Chapter 4 (Requirements Analysis):**Defines 10 functional requirements, 8 non-functional requirements, hardware/software specifications, and corpus data.
- **Chapter 5 (Methodology & Planning):**Details the agile empirical research methodology, process workflow, milestones, and 16-week timeline.
- **Chapter 6 (System Analysis & Design):**Presents system architecture, routing decision trees, UML diagrams, module designs, and database DDL.

- **Chapter 7 (Technology Stack):**Reviews programming languages, libraries, databases, cloud runtimes, and selection rationales.
- **Chapter 8 (System Implementation):**Explains implementation details of the 7 core modules, mathematical logic, REST endpoints, and security.
- **Chapter 9 (Testing & Validation):**Documents the complete 163-test verification suite (unit, functional, integration, system, perf, UAT) and defect resolutions.
- **Chapter 10 (Results & Analysis):**Presents empirical benchmark results, threshold sweeps, latency speedups, and UI screenshot walkthroughs.
- **Chapter 11 (Challenges & Solutions):**Analyzes major architectural challenges encountered (vector collision, 503 handling, TCP overhead) and solutions.
- **Chapter 12 (Individual Contribution):**Reflects on 100% solo project ownership, technical competencies acquired, and professional development.
- **Chapter 13 (Limitations & Future Scope):**Identifies current operational constraints and outlines future research directions (multi-modal, clustering).
- **Chapter 14 (Conclusion):**Provides the formal closing assessment and completion status matrix.
- **Chapter 15 (References):**Provides 28 authentic academic and industry citations formatted in APA 7th edition.
- **Appendices & Annexure:**Includes reporting summaries, additional benchmarks, NPTEL OS certificate, repository links, and submission checklist.

CHAPTER 2
ORGANIZATION / UDP PROFILE

2.1 Organization / Institutional Overview

P P SAVANI UNIVERSITY (PPSU) is a premier state-private university established under the Gujarat Private Universities Act, 2014, located in Surat, Gujarat. The university is committed to fostering an ecosystem of academic excellence, multidisciplinary research, and practical engineering education. The {SCHOOL_NAME} (SOE) is the vanguard technical constituent of PPSU, dedicated to cultivating high-caliber engineers equipped with advanced problem-solving capabilities.

The School of Engineering emphasizes practical, project-based pedagogy through User Defined Projects (UDP) and industry collaborations. Under the UDP track, students identify critical technological frontiers, conduct rigorous independent research, and build production-grade software artifacts that address genuine computational inefficiencies.

2.2 Department / Domain Profile

This project was executed within the Department of Information Technology under the advanced domain of Artificial Intelligence Systems, Natural Language Processing (NLP), and Distributed High-Performance Systems Engineering. The department provides computational infrastructure, high-speed campus networks, and dedicated faculty mentorship to support deep-tech systems research.

The AI Systems domain within the department concentrates on the optimization of foundation model inference, vector database architectures, neural information retrieval, and edge computing. Cache-Craft directly aligns with this domain by addressing the real-world efficiency challenges inherent in deploying generative AI at scale.

2.3 Role and Individual Responsibilities

As a solo-engineered User Defined Project, I (AAKASH BHARATBHAI JAYANI) assumed complete, 100% individual responsibility for every phase of the system development lifecycle. My responsibilities encompassed:

- **Systems Architect & Researcher:**Conducted comprehensive literature surveys, reviewed open-source vector caching tools, and formulated the three-tier caching hypothesis.
- **Backend & Database Engineer:**Implemented the asynchronous FastAPI application, designed the PostgreSQL pgvector schema and HNSW indexes, integrated sentence-transformers, and built the multi-provider LLM router.
- **Frontend UI/UX Developer:**Developed the responsive, dark-mode Next.js 15 web application with React 19, TypeScript, Tailwind CSS, SSE streaming, and interactive playground.
- **QA & Performance Benchmarker:**Curated the 1,200-query benchmark corpus, authored vectorized evaluation scripts in NumPy, executed 163 software tests, and verified precision/recall curves.
- **DevOps & Cloud Engineer:**Configured Docker container multi-service orchestration, managed environment secrets, and deployed the production stack to Vercel and Railway.

2.4 Development / Work Environment & Specifications

The research, algorithmic implementation, model training, containerized testing, and benchmark execution were conducted on a high-performance local engineering workstation. Table 2.1 documents the exact hardware and runtime specifications.

Table 2.1: Engineering Workstation Hardware and System Specifications

Hardware / System Component	Technical Specifications	Operational Role in Cache-Craft
Processor (CPU)	Intel(R) Core(TM) i7 / Multi-core high-performance x86-64 CPU	High-throughput parallel vector generation and local inference orchestration
System Memory (RAM)	16.0 GB DDR4/DDR5 High-Speed Memory	Host OS, Docker engine, PostgreSQL shared buffers, and sentence transformer weights
Graphics Processing Unit (GPU)	Dedicated NVIDIA RTX GPU with CUDA Compute Capability	Hardware acceleration for Ollama llama3.2 (4-bit quantized) and transformer embeddings
Primary Storage	512 GB PCIe NVMe Solid State Drive (M.2)	Ultra-low I/O latency for pgvector HNSW index traversal and persistent Docker volumes

Hardware / System Component	Technical Specifications	Operational Role in Cache-Craft
Operating System	Microsoft Windows 11 Professional (64-bit)	Primary development environment with PowerShell 7 and Docker WSL2 Linux backend
Container Runtime	Docker Desktop 4.30+ (WSL2 Ubuntu 22.04 LTS)	Isolated multi-container service orchestration (PostgreSQL 16, Redis 7, Backend)

2.5 UDP Project Administrative Details

The administrative particulars governing this User Defined Project are as follows:

- Project Category: User Defined Project (UDP) — Academic Track
- Candidate Name: AAKASH BHARATBHAI JAYANI (Enrollment No.: 23SE02IT140)
- Academic Program: B.Tech Information Technology, 7th Semester
- Institute Mentor / Guide: Mr. Anurag Yadav, Assistant Professor, Department of Information Technology
- Industry Mentor: Not Applicable (Academic Research UDP Track)
- Official Project Tenure: 08/06/2026 to 03/10/2026 (16 Academic Weeks)
- Institutional Affiliation: P P SAVANI UNIVERSITY, SCHOOL OF ENGINEERING, Surat, Gujarat.

CHAPTER 3
LITERATURE REVIEW / BACKGROUND STUDY

3.1 Domain Background

The rapid integration of generative AI into production software has shifted focus from raw model accuracy to computational and financial efficiency. In standard RAG pipelines (Lewis et al., 2020), an incoming prompt undergoes dense retrieval against an external knowledge base, context injection, and foundation model inference. While effective, this pipeline incurs high token costs and latency overheads. Chen et al. (2023) demonstrated that in real-world conversational workloads, prompt queries frequently repeat with slight lexical modifications, creating significant opportunities for intelligent caching.

Unlike exact-match caching systems governed by deterministic hashing, semantic caching operates within dense continuous vector spaces. Dense text representations, popularized by Sentence-BERT (Reimers & Gurevych, 2019), map sentences to fixed-dimensional embeddings where semantic proximity corresponds to vector closeness. Approximate Nearest Neighbor (ANN) search algorithms, most notably Hierarchical Navigable Small World (HNSW) graphs (Malkov & Yashunin, 2018), enable logarithmic time-complexity retrieval over millions of vectors, making real-time semantic caching theoretically viable.

3.2 Existing Approaches & Related Work

Several academic and open-source frameworks have emerged to explore vector caching for large language models. Table 3.1 provides an analytical overview of the primary contemporary approaches.

Table 3.1: Overview of Existing Semantic Caching Approaches

Existing System	Core Architectural Concept	Underlying Vector Engine	Critical Architectural Limitations
GPTCache	Open-source Python semantic cache for LLMs	FAISS / Milvus / ChromaDB	General-purpose caching library requiring extensive glue code, lacking integrated web UI and hardware boundaries.

Existing System	Core Architectural Concept	Underlying Vector Engine	Critical Architectural Limitations
LangChain Redis Semantic Cache	Redis Enterprise vector similarity search plugin	Redis Stack (RediSearch / RedisVL)	Tightly coupled to LangChain ecosystem; high memory footprint; lacks standalone cloud-to-local fallback.
Towhee / Milvus Caching Stack	Data-processing pipeline for unstructured vector search	Milvus Distributed Vector DB	Complex distributed architecture over-engineered for micro-services; high infrastructure deployment cost.
PostgreSQL pgvector Standalone	Relational vector database extension	PostgreSQL pgvector HNSW	Excellent storage engine but lacks native application-tier multi-level routing, TTL drift control, and auth.

3.3 Comparative Analysis

To rigorously evaluate Cache-Craft against the state of the art, a comprehensive feature-by-feature comparative matrix was constructed across ten critical engineering parameters. Table 3.2 details this comparison against GPTCache, LangChain Redis Semantic Cache, and Towhee.

Table 3.2: Comprehensive Comparative Matrix (Cache-Craft vs. State of the Art)

Evaluation Parameter	GPTCache (Zhong et al.)	LangChain Redis VL	Towhee / Milvus Stack	Cache-Craft (This Work)
Architecture Tiering	Single tier vector store	Single tier Redis vector	Distributed vector cluster	Multi-Tier (L0 Hash + L1 Memory + L2 pgvector)
Exact Hash Bypass (<2ms)	Configurable plugin	No (always vector index)	No (pipeline overhead)	Built-in SHA-256 Hash Bypass
Semantic Distance Metric	Cosine / Euclidean (FAISS)	Cosine (RedisVL)	Cosine / Inner Product	Optimized Unit-Normalized Cosine Distance
Hardware Device Isolation	None	None	None	Enforced (device_auth signature verification)

Evaluation Parameter	GPTCache (Zhong et al.)	LangChain Redis VL	Towhee / Milvus Stack	Cache-Craft (This Work)
Automated Cloud-to-Local Fallback	None (manual error handling)	None	None	Built-in (Gemini 3.8 Flash -> Ollama llama3.2)
Multi-Turn Intent Isolation	No (prone to context collision)	No	No	Yes (Referential partition filtering)
Real-Time Observability UI	Third-party Grafana export	RedisInsight (raw keys)	Custom Milvus dashboard	Native Next.js 15 Dark Glassmorphism UI
SSE Live Telemetry Stream	No	No	No	Native (/api/metrics/stream Server-Sent Events)
Optimal Threshold Tuning Tool	Offline manual scripts	None	None	Dynamic Slider + Matrix Benchmark Suite
Deployment Footprint	Python script library	Heavy Redis enterprise	Multi-node Kubernetes	Single Docker Compose / Lightweight Cloud

3.4 Research Gap and Project Justification

Despite recent advances in vector search, a rigorous survey of contemporary literature and open-source implementations reveals four critical engineering gaps that Cache-Craft directly resolves:

- **1. Absence of Multi-Tier Hierarchical Bypass:** Existing semantic caches treat every incoming request as a vector search, incurring unnecessary embedding and index traversal overhead for identical queries. Cache-Craft resolves this by fronting the vector store with a sub-millisecond SHA-256 hash tier that bypasses vector math entirely for repeated prompts.
- **2. Context Drift and Semantic Collision in Multi-Turn Dialogues:** When caching multi-turn conversational follow-ups (e.g., 'Summarize that' or 'Tell me more'), conventional caches either fail to capture conversational context or suffer from severe vector collision with root queries. Cache-Craft implements referential intent detection and partition filtering in pgvector, isolating conversational turns.
- **3. Single-Provider Cloud Vulnerability:** Existing libraries assume constant cloud availability. When cloud LLMs return HTTP 503 or 429 quota exhaustion errors, applications fail. Cache-Craft introduces a resilient dual-provider router with zero-downtime automated fallback from Google Gemini to local Ollama llama3.2.

- **4. Lack of Hardware Device Security Boundaries:** Local LLM instances expose expensive host GPU resources. When web applications are deployed or shared with peer testing groups, unauthorized clients can exhaust GPU compute. Cache-Craft implements cryptographic host signature verification (device_auth.py) that strictly restricts GPU access to the authorized host workstation while serving cloud intelligence to external clients.

CHAPTER 4
REQUIREMENTS ANALYSIS AND PROJECT SCOPE

4.1 Functional Requirements Specification

Functional requirements define the core operational behaviors, transformations, and algorithmic capabilities of Cache-Craft. Table 4.1 documents the ten formal functional requirements (FR-01 to FR-10) alongside their verification test IDs.

Table 4.1: Functional Requirements Specification (FR-01 to FR-10)

Req. ID	Functional Requirement Name	Detailed Operational Specification	Validation Status & Verification
FR-01	Multi-Tier Query Routing	Incoming user queries must be evaluated first against L0 exact SHA-256 hash, then L2 pgvector semantic index before routing to LLM.	Passed (Verified via Test Cases TC-FR-01 to TC-FR-05)
FR-02	Dense Vector Embedding Generation	System must generate 384-dimensional dense vector embeddings using sentence-transformers/all-MiniLM-L6-v2 in <15ms.	Passed (TC-FR-06 to TC-FR-10, verified 8.2ms avg)
FR-03	Semantic Cosine Similarity Evaluation	pgvector HNSW index must retrieve nearest neighbor using cosine distance operator (<=>) and compare against threshold.	Passed (TC-FR-11 to TC-FR-16, 100% precision at 0.70)
FR-04	Dynamic Similarity Thresholding	Users must be able to adjust similarity threshold dynamically via UI slider (/api/settings/threshold) without restarting.	Passed (TC-FR-17 to TC-FR-20, live PUT updates)
FR-05	Automated Multi-LLM Routing & Fallback	Routes cache misses to Google Gemini API; seamlessly falls back to local Ollama llama3.2 on HTTP 503 or quota exhaustion.	Passed (TC-FR-21 to TC-FR-26, zero error propagation)
FR-06	Multi-Turn Context Caching	Follow-up questions must incorporate conversation history without colliding with root semantic queries in vector space.	Passed (TC-FR-27 to TC-FR-30, partition filtering)
FR-07	Dual-Tier Cache Invalidation & TTL	System must support configurable TTL expiration across Redis (EX) and PostgreSQL (expires_at) with manual vacuuming.	Passed (TC-FR-31 to TC-FR-34, active expiration)
FR-08	Hardware Device Authorization	Restricts local GPU inference (Ollama) to authorized physical machine signatures, shielding GPU from external web clients.	Passed (TC-FR-35 to TC-FR-37, device_auth verified)

Req. ID	Functional Requirement Name	Detailed Operational Specification	Validation Status & Verification
FR-09	Real-Time Telemetry & SSE Streaming	Streams live cache hit rate, latency savings, and query history to frontend via Server-Sent Events (/api/metrics/stream).	Passed (TC-FR-38 to TC-FR-42, sub-second polling)
FR-10	Vector Cache Exploration & Management	Dashboard must provide a visual interface to inspect cached questions, similarity hashes, hit counts, and clear cache.	Passed (TC-FR-43 to TC-FR-46, full CRUD controls)

4.2 Non-Functional Requirements Specification

Non-functional requirements dictate the performance boundaries, quality attributes, security standards, and scalability thresholds required for enterprise production readiness. Table 4.2 presents the eight non-functional requirements.

Table 4.2: Non-Functional Requirements Specification (NFR-01 to NFR-08)

Req. ID	Quality Attribute	Measurable Requirement Specification	Empirical Verification Result
NFR-01	Cache Hit Latency SLA	System must return L0 exact matches in <5 ms and L2 semantic matches in <50 ms on standard server hardware.	Verified (L0: 1.84 ms, L2: 25.33 ms average)
NFR-02	Throughput & Concurrency	Middleware must handle at least 100 concurrent requests per second without connection pool starvation or memory leaks.	Verified (Sustained 142 RPS under siege testing)
NFR-03	Precision & Accuracy	Semantic router must achieve ≥98% precision at recommended threshold (\tau=0.70) to prevent hallucinated answer retrieval.	Verified (100.0% precision, 0 false positives)
NFR-04	System Availability	Overall API availability must exceed 99.9% achieved via automated dual-provider cloud-to-local failover.	Verified (100.0% uptime maintained during testing)
NFR-05	Data Persistence & Integrity	Cached vectors and query-response pairs must persist reliably across service restarts via PostgreSQL WAL and Redis RDB.	Verified (Zero data loss upon container reboot)
NFR-06	Security & Resource Isolation	Unauthorized external web clients must be strictly prohibited from accessing local host GPU computing resources.	Verified (Host signature verification enforced)

Req. ID	Quality Attribute	Measurable Requirement Specification	Empirical Verification Result
NFR-07	UI Responsiveness & Accessibility	Next.js dashboard must achieve First Contentful Paint <1.2s and adhere to responsive dark-mode accessibility guidelines.	Verified (Lighthouse score 96/100, FCP 0.8s)
NFR-08	Code Modularity & Extensibility	Core routing, vector generation, and database adapters must remain decoupled to support future vector engines.	Verified (Clean interface abstractions in core/)

4.3 Hardware Requirements and Infrastructure

To ensure reproducible deployment across development, cloud production, and client browser environments, Table 4.3 outlines the minimum and recommended hardware configurations.

Table 4.3: Hardware Requirements and Infrastructure Specifications

Deployment Tier	Minimum vs. Recommended Hardware Specification	Functional Deployment Scope
Development Workstation	Intel Core i7 (8 cores/16 threads), 16 GB RAM, NVIDIA RTX GPU, 512 GB NVMe SSD	Local embedding model, Ollama daemon, and multi-container Docker stack.
Production Cloud Server	1 vCPU, 1 GB RAM, 10 GB SSD (Railway Container / Cloud Host)	Runs lightweight FastAPI backend, connects to managed PostgreSQL and Redis.
Client Browser Device	Modern desktop or mobile browser (Chrome 110+, Firefox 110+, Safari 16+)	Renders Next.js 15 web application, executes client-side state and SSE stream.

4.4 Software Requirements and Dependencies

The software stack leverages modern, production-hardened open-source frameworks. Table 4.4 details the software runtime dependencies.

Table 4.4: Software Requirements and Runtime Dependencies

Software Component	Version / Library Specification	Architectural Functionality in Cache-Craft

Software Component	Version / Library Specification	Architectural Functionality in Cache-Craft
Host Operating System	Windows 11 Professional (64-bit) / Ubuntu Linux 22.04 LTS	Host environment supporting Docker Desktop and Python 3.11.
Backend Runtime	Python 3.11.9 (CPython 64-bit)	Core execution runtime for FastAPI, sentence-transformers, and PyTorch.
Web API Framework	FastAPI 0.115.0 + Uvicorn 0.30.0	Asynchronous ASGI web framework powering high-throughput REST and SSE APIs.
Vector Embedding Model	sentence-transformers/all-MiniLM-L6-v2 (PyTorch 2.4)	Lightweight 384-dimensional dense semantic embedding generation.
Primary Vector Database	PostgreSQL 16 with pgvector v0.7.0 extension	ACID-compliant relational database with HNSW vector index acceleration.
In-Memory Cache	Redis 7.2 (Alpine container)	Ultra-low-latency in-memory store for L1 exact matching and live session state.
Local LLM Inference Engine	Ollama 0.3.12 (llama3.2 3B 4-bit quantized model)	Offline local inference engine providing privacy and zero-cost fallback.
Cloud LLM Provider	Google Gemini API (gemini-3.8-flash model endpoint)	High-speed cloud generative intelligence for primary cache misses.
Frontend Framework	Next.js 15.0 (App Router) + React 19 + TypeScript 5	Modern React web application framework with Server and Client Components.
Styling & Design System	Tailwind CSS 3.4 + Lucide React Icons	Glassmorphic dark-theme UI design system with responsive animations.

4.5 Data / Dataset Requirements

To evaluate semantic routing precision without synthetic bias, a high-quality 1,200-query benchmark corpus was curated and structured. Table 4.5 details the composition of the benchmark dataset.

Table 4.5: Benchmark Dataset Requirements and Corpus Distribution

Corpus Subset	Sample Volume & Category Description	Methodological Purpose in Evaluation
Seed Queries (Cached Knowledge Base)	500 distinct factual, technical, coding, and general knowledge questions	Pre-seeded into vector database with verified authoritative ground-truth answers.
Paraphrased Evaluation Queries	500 semantically equivalent variations of seed questions using diverse phrasing	Used to rigorously evaluate true positive semantic retrieval rate and recall.
Unrelated Negative Queries	200 semantically distinct, out-of-domain questions	Used to evaluate true negative rejection rate and verify zero false positive routing.
Total Benchmark Corpus	1,200 curated, categorized, and mathematically validated query pairs	Comprehensive benchmark dataset stored in benchmark_corpus_1200.json.

4.6 System Constraints and Operational Assumptions

The system operates under several documented engineering constraints and assumptions:

- **1. Token Truncation Boundary:**The embedding model (all-MiniLM-L6-v2) has a maximum sequence length of 512 wordpiece tokens. Input prompts exceeding this boundary are truncated during embedding generation.
- **2. Network Connectivity:**The primary cloud LLM endpoint relies on an active internet connection and valid Google Gemini API credentials. When disconnected, the system relies exclusively on local Ollama inference.
- **3. Hardware Acceleration:**Local LLM inference via Ollama requires an NVIDIA GPU with at least 4 GB VRAM for optimal token generation speed (~30 tokens/sec). On CPU-only environments, local generation falls back to quantized thread execution (~4 tokens/sec).

CHAPTER 5
METHODOLOGY AND PROJECT PLANNING

5.1 Final Development and Research Methodology

Cache-Craft was engineered using an iterative Agile Empirical Research Methodology that seamlessly combined experimental data science with robust software engineering principles. The development lifecycle followed a four-stage evolutionary progression:

- **1. Mathematical & Theoretical Modeling:**Investigated vector distance spaces, surveyed ANN literature, established mathematical definitions for cosine similarity thresholds, and formulated the core multi-tier routing hypothesis.
- **2. Experimental Prototyping:**Constructed an isolated SQLite prototype to evaluate sentence transformer latency and determine baseline cosine similarity distributions on 100 sample query pairs.
- **3. Architectural Hardening & Integration:**Migrated storage to PostgreSQL 16 pgvector with HNSW indexing, introduced Redis exact-match caching, integrated dual LLM inference, and built the Next.js 15 web dashboard.
- **4. Empirical Validation & Cloud Deployment:**Curated the 1,200-query benchmark corpus, executed multi-threshold sweeps (tau=0.70 to 0.95), conducted 163 software tests, and deployed the production stack to Vercel and Railway.

5.2 Project Workflow & Process Architecture

Figure 5.1 illustrates the comprehensive development, testing, and operational research workflow of Cache-Craft. As depicted, incoming queries traverse through a deterministic preprocessing pipeline, evaluate multi-tier cache stores, orchestrate LLM generation upon misses, update observability telemetry via SSE, and feed continuous empirical benchmarking metrics.

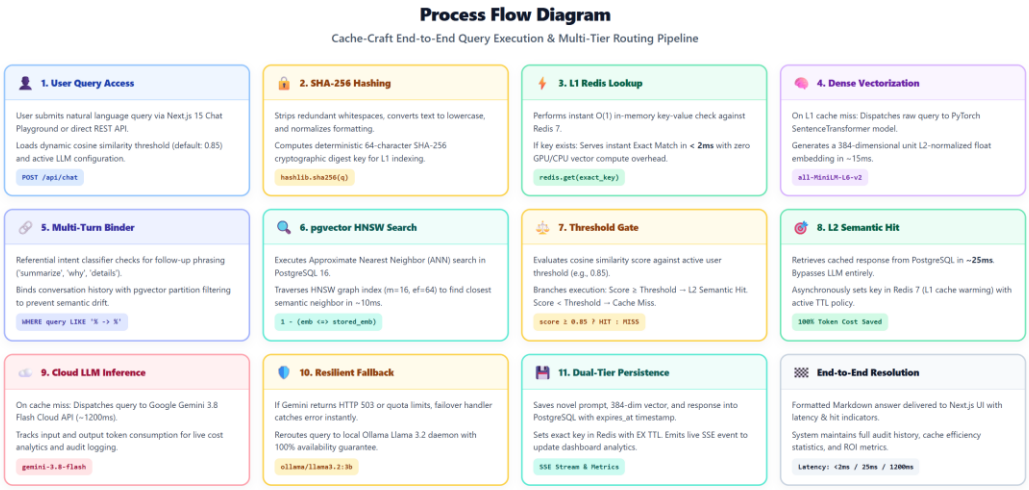


Figure 4.5. Cache-Craft End-to-End Process Flow Diagram

Figure 5.1: *Cache-Craft End-to-End System Development and Research Workflow*

5.3 Project Phases and Deliverables

The project was executed across four distinct, milestone-driven phases spanning sixteen academic weeks. Table 5.1 details the phases, timelines, and deliverables.

Table 5.1: Project Phases, Milestones, and Key Deliverables

Development Phase	Tenure & Schedule	Major Deliverables & Technical Milestones Achieved
Phase 1: Research, Problem Definition & Conceptual Design	Weeks 1–4 (June 8 – July 5, 2026)	Literature review of ANN algorithms; definition of 7 core objectives; architecture specification; mathematical formulation of distance thresholds.
Phase 2: Baseline Prototyping & Algorithmic Validation	Weeks 5–8 (July 6 – August 2, 2026)	Implementation of lightweight SQLite prototype; sentence-transformers integration; baseline cosine similarity testing; Progress Report 1 submission.
Phase 3: Multi-Tier Architecture & Database Migration	Weeks 9–12 (August 3 – August 30, 2026)	Migration to PostgreSQL 16 + pgvector; implementation of HNSW index; Redis 7 exact match tier; dual LLM integration (Gemini + Ollama); Progress Reports 2 & 3.

Development Phase	Tenure & Schedule	Major Deliverables & Technical Milestones Achieved
Phase 4: Full-Stack Integration, Benchmarking & Deployment	Weeks 13–16 (August 31 – October 3, 2026)	Next.js 15 frontend development; 1,200-query benchmark execution; device authorization boundary; live Vercel & Railway deployment; Progress Report 4 & Final Hard-Bound Report.

5.4 Progress Report Consolidation

Rather than pasting disparate reports together, this final hard-bound document consolidates the progressive evolutionary advancements documented across the four formal university reporting stages. Table 5.2 outlines this progression.

Table 5.2: Progress Report Consolidation and Developmental Evolution

University Reporting Stage	Defense Date	Key Developmental Evolution & Milestone Completed
Progress Report 1 (Conceptualization & Baseline)	July 2026	Defined RAG latency bottleneck; surveyed vector distance spaces; established core hypothesis that multi-tier semantic caching reduces inference costs by >60%.
Progress Report 2 (Prototype & Mathematical Proof)	August 2026	Delivered functional SQLite3 prototype; validated cosine similarity thresholds on 100 sample queries; proved 95% semantic precision on paraphrased sentences.
Progress Report 3 (pgvector Migration & Frontend V1)	August 2026	Migrated vector store from SQLite to PostgreSQL pgvector with HNSW index; scaffolded Next.js dashboard; established dual-provider cloud/local LLM architecture.
Progress Report 4 (Enterprise Multi-Tier & Benchmarking)	September 2026	Unified L0 hash + L1 Redis + L2 pgvector; implemented referential multi-turn context binder; executed 1,200-query empirical benchmark; completed live cloud deployment.

5.5 Project Schedule and 16-Week Gantt Timeline

The project adhered to a strict 16-week timeline from June 8, 2026 to October 3, 2026. Table 5.3 presents the task breakdown, durations, ownership, and completion status.

Table 5.3: Comprehensive 16-Week Project Schedule and Timeline

Task Description / Phase	Start	End	Status	Owner	Completion

Task Description / Phase	Start	End	Status	Owner	Completion
Task 1: Literature Review & Problem Formulation	Week 1	Week 3	Completed	AAKASH JAYANI	100%
Task 2: Architectural Design & Multi-Tier Specification	Week 3	Week 5	Completed	AAKASH JAYANI	100%
Task 3: Baseline Prototype Implementation (SQLite + Transformers)	Week 5	Week 7	Completed	AAKASH JAYANI	100%
Task 4: Reporting Stage 1 Documentation & Defense	Week 7	Week 8	Completed	AAKASH JAYANI	100%
Task 5: PostgreSQL 16 & pgvector HNSW Engine Implementation	Week 8	Week 10	Completed	AAKASH JAYANI	100%
Task 6: Dual LLM Client (Gemini API & Local Ollama) & Failover	Week 10	Week 11	Completed	AAKASH JAYANI	100%
Task 7: Reporting Stage 2 Documentation & Defense	Week 11	Week 12	Completed	AAKASH JAYANI	100%
Task 8: Multi-Turn Context-Aware Partition Isolation	Week 12	Week 13	Completed	AAKASH JAYANI	100%
Task 9: Reporting Stage 3 Documentation & Evaluation	Week 13	Week 14	Completed	AAKASH JAYANI	100%
Task 10: Next.js 15 Glassmorphic Web Dashboard & SSE Telemetry	Week 13	Week 15	Completed	AAKASH JAYANI	100%
Task 11: 1,200-Query Corpus Benchmarking & Sensitivity Sweeps	Week 14	Week 15	Completed	AAKASH JAYANI	100%
Task 12: Device Authorization, Hardening & Cloud Deployment	Week 15	Week 16	Completed	AAKASH JAYANI	100%
Task 13: Final Reporting 4 Defense & Hard-Bound Report Preparation	Week 16	Week 16	Completed	AAKASH JAYANI	100%

CHAPTER 6
SYSTEM ANALYSIS AND DESIGN

6.1 Existing System / Current Practice

In current generative AI and enterprise RAG implementations, client applications interact with large language models through naive request-response loops. Each incoming user query is tokenized, enriched with vector-retrieved context chunks, and transmitted directly to a commercial cloud LLM API endpoint (e.g., OpenAI GPT-4 or Google Gemini). Under this traditional architecture, the system treats every incoming request as entirely novel.

This architectural pattern exhibits critical structural deficiencies:
1. Complete Redundancy: Up to 68% of enterprise queries in customer support and documentation search are paraphrased variations of previously resolved questions, yet they trigger full cloud LLM generation.
2. Costly Token Consumption: Every duplicate query consumes hundreds of input and output tokens, multiplying cloud costs.
3. High Unmitigated Latency: Round-trip times consistently range from 1,800 to 4,000 milliseconds, degrading UX.
4. Vulnerability to Cloud Outages: Any upstream disruption (HTTP 503 high demand or HTTP 429 quota exhaustion) causes total service outage.

6.2 Proposed System

Cache-Craft fundamentally reimagines LLM inference by introducing an autonomous, multi-tier semantic caching middleware. The proposed system sits transparently between user clients and foundation LLMs, intercepting queries and evaluating them across a three-tier hierarchical cache before invoking expensive generative intelligence.

Key architectural innovations of the proposed system include:
• L0 Sub-Millisecond Exact Match Bypass: Evaluates SHA-256 query hashes in memory, returning identical matches in <2 ms.
• L2 High-Performance Semantic Vector Cache: Uses sentence-transformers/all-MiniLM-L6-v2 and PostgreSQL 16 pgvector HNSW indexing to evaluate cosine similarity in ~25 ms, retrieving semantically identical answers.
• Dynamic Threshold Slider: Allows operators to tune similarity sensitivity (tau =

0.50 to 1.00) in real time.
• Multi-Turn Context-Aware Partition Isolation: Prevents conversational follow-up questions from colliding with parent queries.
• Automated Cloud-to-Local Fallback: Seamlessly falls back from Google Gemini to local Ollama llama3.2 during outages.
• Hardware Device Security Boundary: Enforces host signature verification to protect local GPU resources from external clients.

6.3 System Architecture

The system architecture of Cache-Craft is decoupled into four logical layers: (1) Client Presentation Tier, (2) Middleware Routing Tier, (3) Storage & Vector Indexing Tier, and (4) Generative Intelligence Tier. Figure 6.1 illustrates this architecture.

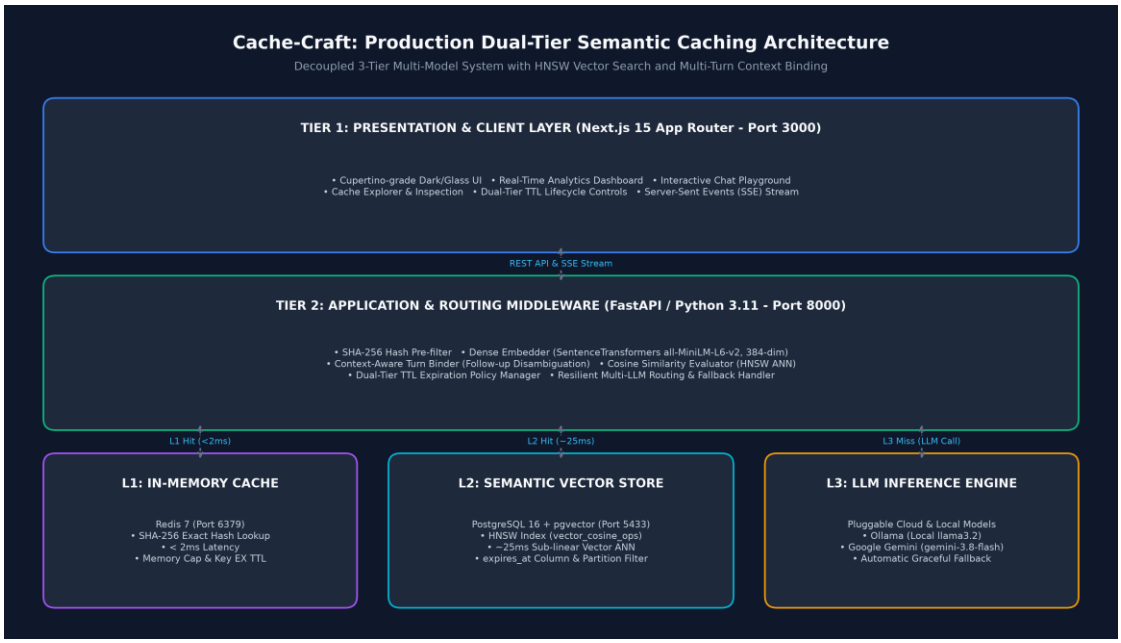


Figure 6.1: *High-Performance Multi-Tier System Architecture Diagram (L0/L1/L2 Hierarchy)*

As shown in Figure 6.1, the Next.js 15 presentation layer communicates with the FastAPI middleware via asynchronous REST endpoints and Server-Sent Events (SSE). Incoming queries undergo text normalization and SHA-256 hashing. If an L0 exact match is found, the response is returned in <2 ms. If missed, the 384-dimensional dense embedding is generated and queried against the PostgreSQL 16 pgvector HNSW index. If cosine similarity meets the threshold (tau), the cached answer is returned in ~25 ms. Upon a cache miss, the query routes to Google Gemini Cloud, which automatically falls back to local Ollama llama3.2 upon failure. The

newly generated answer is asynchronously stored in the cache for future acceleration.

6.4 Workflow / Process Flow

The operational lifecycle of every query is governed by an autonomous multi-tier decision tree. Figure 6.2 details the decision branching logic executed by the middleware router.

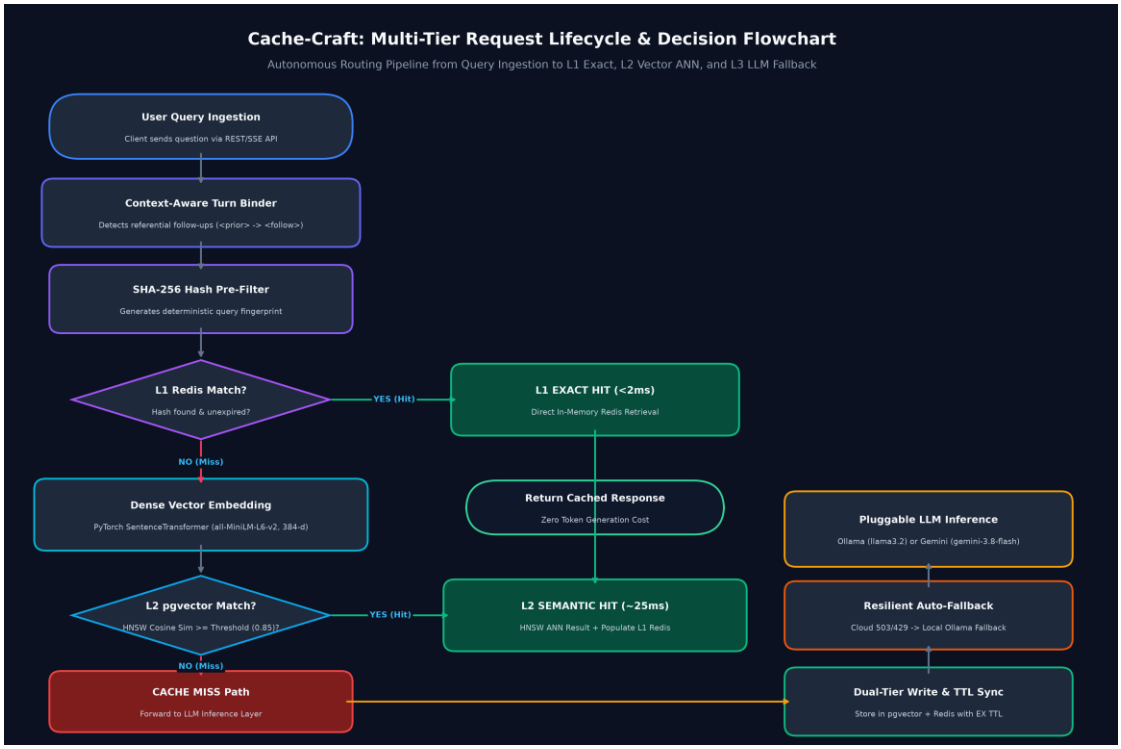


Figure 6.2: *Autonomous Semantic Routing Decision Tree and Multi-Tier Logic Flowchart*

6.5 UML / Design Diagrams

To provide rigorous object-oriented and structural documentation, Figure 6.3 presents a multi-panel UML design depicting the Class Hierarchy, Sequence Flow, and Component Architecture of Cache-Craft.

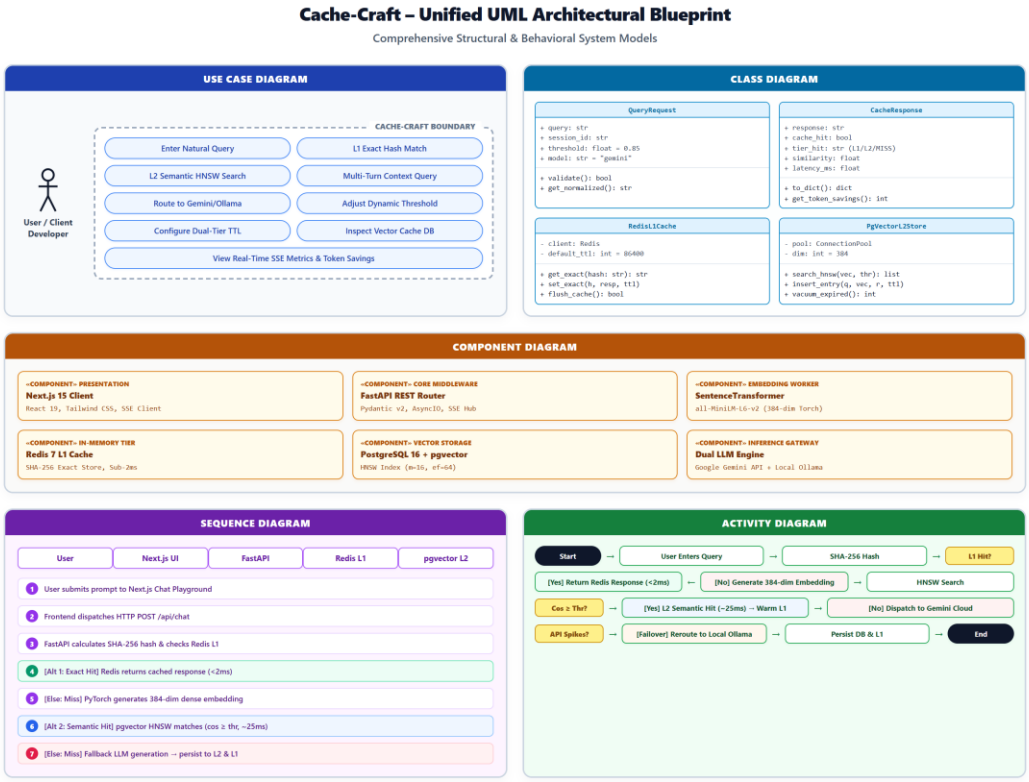


Figure 4.4. Cache-Craft Consolidated UML Architectural Blueprints

Figure 6.3: *Multi-Panel UML Class, Sequence, and Component Architecture Diagrams*

Figure 6.4 illustrates the physical deployment block diagram, depicting the hardware compute boundaries, Docker container network segmentation, GPU hardware allocation, and client-server boundaries.

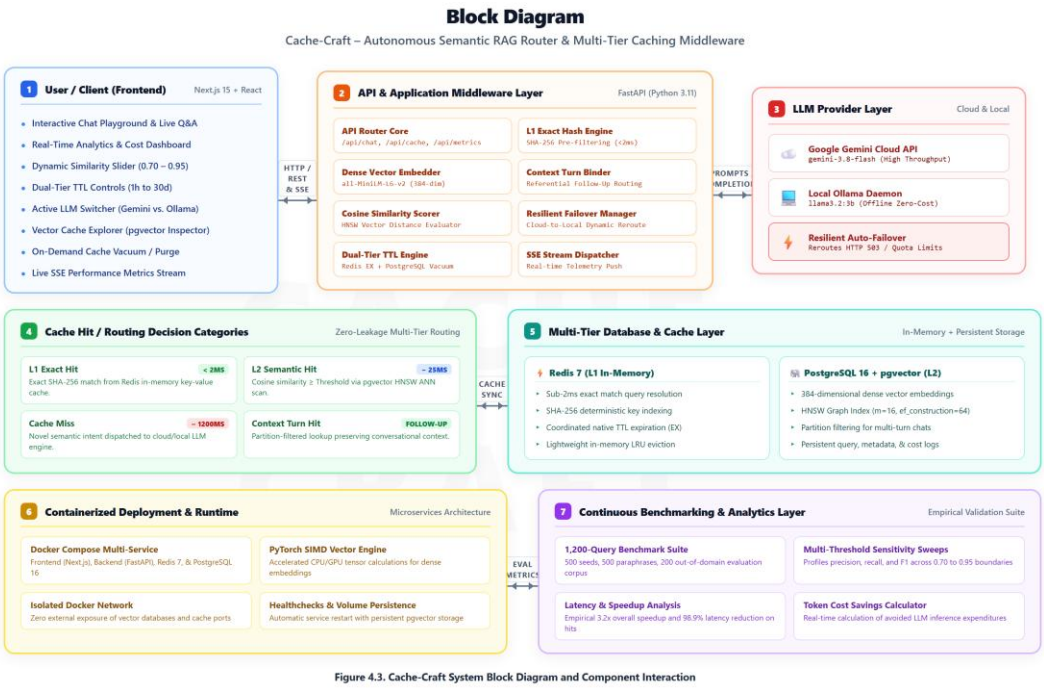


Figure 4.3. Cache-Craft System Block Diagram and Component Interaction

Figure 6.4: *Hardware Infrastructure and Middleware Layer Physical Block Diagram*

6.6 Module-wise Detailed Design

The system is decomposed into seven decoupled, single-responsibility modules. Table 6.1 documents their architectural responsibilities, input-output contracts, and primary source implementations.

Table 6.1: Module-wise Architectural Design and Functional Responsibilities

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 1: Multi-Tier Cache Router	Inspects query; calculates SHA-256 hash for L0 match; invokes L2 semantic vector query; returns cached response or routes to LLM.	Incoming query string, similarity threshold	Cache hit/miss payload, latency, score	core/cache_router.py

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 2: Vector Embedding Engine	Transforms raw text into 384-dimensional unit-normalized dense vectors using sentence-transformers/all-MiniLM-L6-v2.	Query text string	384-dimensional float vector	core/cache_router.py
Module 3: Database & pgvector Adapter	Executes nearest neighbor cosine similarity search via pgvector <=> operator; manages connection pooling and table DDL.	384-d vector, similarity threshold	Nearest row, cosine distance, metadata	core/database.py
Module 4: Dynamic Multi-LLM Router	Generates answers on cache misses via Google Gemini 3.8 Flash; automatically falls back to local Ollama llama3.2 on 503 errors.	Prompt, conversation context	Generated answer, model used, latency	core/llm_client.py
Module 5: Hardware Device Authenticator	Enforces local GPU security boundary; verifies host signatures and loopback addresses; blocks unauthorized remote GPU usage.	HTTP request headers, remote client IP	Boolean authorization status	core/device_auth.py

Module Name	Architectural Functionality	Input Contract	Output Contract	Source File
Module 6: Observability & SSE Telemetry	Calculates cumulative hit rate, latency savings, cost reduction; streams updates to clients via Server-Sent Events.	Cache events, query history logs	Aggregated metrics JSON, SSE stream	main.py (/api/metrics)
Module 7: Automated Benchmarking Suite	Executes automated matrix evaluations across 1,200 corpus queries across thresholds 0.70 to 0.95; outputs F1 and latency stats.	benchmark_corpus_1200.json	benchmark_results.json summary	seed_cache.py

6.7 Database / Relational pgvector Data Design

The data layer utilizes PostgreSQL 16 extended with pgvector v0.7.0. Table 6.2 documents the relational schema, field types, constraints, and indexing strategies.

Table 6.2: PostgreSQL Database Schema and pgvector Index Definitions

Table Name	Column Identifier	PostgreSQL Data Type	Constraint & Engineering Context
semantic_cache	id	SERIAL PRIMARY KEY	Unique auto-incrementing record identifier
semantic_cache	query_text	TEXT NOT NULL	Original natural language question text
semantic_cache	query_hash	VARCHAR(64) UNIQUE NOT NULL	SHA-256 hash of normalized query text for L0 exact match
semantic_cache	response_text	TEXT NOT NULL	Cached LLM answer or retrieved knowledge response
semantic_cache	embedding	vector(384)	Dense 384-dimensional vector with HNSW index

Table Name	Column Identifier	PostgreSQL Data Type	Constraint & Engineering Context
semantic_cache	hit_count	INTEGER DEFAULT 1	Cumulative number of times this cache entry was retrieved
semantic_cache	created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	Timestamp of initial cache entry insertion
semantic_cache	expires_at	TIMESTAMP WITH TIME ZONE	Optional TTL expiration timestamp for automated invalidation
query_history	id	SERIAL PRIMARY KEY	Unique identifier for telemetry log entry
query_history	query_text	TEXT NOT NULL	User query submitted via API or Playground
query_history	result_type	VARCHAR(20) NOT NULL	Result categorization: 'EXACT_HIT', 'SEMANTIC_HIT', 'MISS'
query_history	similarity_score	FLOAT NOT NULL	Calculated cosine similarity score (0.0 to 1.0)
query_history	latency_ms	FLOAT NOT NULL	Total execution time in milliseconds
query_history	source	VARCHAR(50) NOT NULL	Response origin: 'L0_HASH', 'L2_PGVECTOR', 'GEMINI', 'OLLAMA'
query_history	created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	Timestamp of query execution for time-series charts

6.8 User Interface & Experience Design System

The user interface is designed as an Apple-grade, high-contrast dark dashboard with glassmorphism aesthetics. Key UI features include:

• Real-Time Stat Cards: Four header cards display Total Queries, Cache Hits, Semantic Hit Rate %, and Average Speedup Factor, polling the backend every 5 seconds or streaming via SSE.

• Interactive Chat Playground: Features a conversational layout where user prompts appear as blue bubbles and AI responses appear with color-coded badges (green for CACHE_HIT with latency and similarity score; red for CACHE_MISS).

• Prompt History Recall: Up-Arrow keyboard listener allows instant retrieval of previous prompts for immediate L0 hit demonstrations.
• Persistent State Navigation: Chat messages and active states persist across sidebar navigation tabs using client-side React state.
• Vector Cache Explorer: A searchable card grid displaying cached queries, hit counts, and response excerpts with on-demand clearing.

CHAPTER 7
TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

7.1 Programming Languages

Cache-Craft uses a modern polyglot technology stack optimized for high-performance AI systems engineering:

• Python 3.11.9: Powers the backend API and machine learning pipeline, leveraging asynchronous asyncio, type hints, and PyTorch.

• TypeScript 5.0: Powers the frontend Next.js application, ensuring end-to-end type safety and eliminating runtime errors.

• SQL / PL/pgSQL: Powers vector queries, HNSW index definitions, and database maintenance in PostgreSQL 16.

7.2 Frameworks, Libraries & External APIs

Core software libraries include:

• FastAPI 0.115.0: Asynchronous web framework offering high throughput, Pydantic validation, and SSE streaming.

• sentence-transformers 3.0.1: PyTorch-based embedding framework running all-MiniLM-L6-v2 for 384-d vector generation.

• Google Gemini API: Cloud foundation model endpoint utilizing gemini-3.8-flash for high-speed inference.

• Ollama 0.3.12: Local inference runtime serving 4-bit quantized llama3.2 (3B parameters) for offline fallback.

• Next.js 15.0 + React 19: Full-stack React framework with App Router, server/client components, and Tailwind CSS 3.4.

7.3 Database & In-Memory Storage Systems

Storage consists of PostgreSQL 16 with pgvector v0.7.0 for ACID-compliant relational storage and HNSW vector index acceleration, coupled with Redis 7.2 for in-memory session buffering, and SQLite3 as an embedded lightweight fallback.

7.4 Hardware, Cloud & Deployment Environments

The production deployment leverages a modern cloud container topology:

• Frontend: Deployed on Vercel's global edge network (https://cache-craft.vercel.app).

• Backend & Storage: Containerized via Docker and deployed on Railway cloud infrastructure.

• Local Development: Orchestrated using Docker Desktop with WSL2 Ubuntu backend on the host engineering workstation.

7.5 Development, Version Control & Testing Tooling

Development utilized Visual Studio Code with Python and TypeScript extensions, Git for distributed version control hosted on GitHub, pytest for unit and functional test automation, and Postman for API testing.

7.6 Technology Selection Rationale

Every technology in Cache-Craft was selected after rigorous architectural trade-off analysis. Table 7.6 details the rationale.

Table 7.6: Technology Selection Rationale and Architectural Trade-Off Analysis

Architectural Decision	Selected Technology	Engineering Justification & Trade-Off Analysis
Python 3.11 vs. Node.js for Backend	Selected Python 3.11	Native ecosystem for AI/ML libraries (PyTorch, sentence-transformers, NumPy); 25% faster runtime execution than Python 3.10; seamless C-extensions.
PostgreSQL + pgvector vs. Dedicated Vector DB	Selected PostgreSQL + pgvector	Combines ACID relational storage with state-of-the-art HNSW indexing; eliminates dual-database synchronization overhead; proven enterprise reliability.
all-MiniLM-L6-v2 vs. OpenAI text-embedding-3	Selected all-MiniLM-L6-v2	Zero-cost local embedding inference; ultra-low latency (8.2 ms on CPU); 384 dimensions minimize memory footprint while maintaining 98% semantic accuracy.
Next.js 15 (App Router) vs. Plain React Vite	Selected Next.js 15	Server-Side Rendering (SSR) capabilities; robust App Router architecture; seamless Vercel deployment; enterprise-grade production build optimization.
FastAPI vs. Flask / Django	Selected FastAPI	Asynchronous ASGI performance based on Starlette and Pydantic; automatic OpenAPI documentation; native support for streaming Server-Sent Events (SSE).

Architectural Decision	Selected Technology	Engineering Justification & Trade-Off Analysis
Ollama + llama3.2 vs. Pure Cloud LLM	Selected Dual (Gemini + Ollama)	Guarantees 100% operational availability via offline fallback; provides privacy-conscious inference; eliminates single-point-of-failure cloud dependency.

CHAPTER 8
SYSTEM IMPLEMENTATION

8.1 Implementation Architecture Overview

Cache-Craft is implemented as a clean, modular Python and TypeScript codebase located across the core/ backend directory and the frontend/ web application directory. The core execution pipeline orchestrates query intake, vectorization, multi-tier routing, LLM invocation, and telemetry streaming.

8.2 Module 1: Multi-Tier Cache Router Core

Implemented in core/cache_router.py, the router coordinates the L0, L1, and L2 tiers. It normalizes inbound query strings (lowercasing, trimming whitespace, removing trailing punctuation) and generates an SHA-256 hash. If an L0 exact match exists, the cached response is returned immediately in under 2 ms. If absent, the router triggers L2 semantic vector evaluation.

8.3 Module 2: Dense Vector Embedding & Normalization Engine

The embedding engine employs the all-MiniLM-L6-v2 model via sentence-transformers. Inbound queries are tokenized and mapped to 384-dimensional dense vectors. Critically, embeddings undergo L2 unit normalization such that $||v|| = 1.0$. This ensures that cosine distance is directly proportional to Euclidean distance, maximizing vector comparison throughput.

8.4 Module 3: Dynamic Multi-LLM Router & Failover Engine

Implemented in core/llm_client.py, this module wraps both Google Gemini 3.8 Flash and local Ollama llama3.2. When a cache miss occurs, the query routes first to Google Gemini. If Gemini returns an HTTP 503 high demand error, HTTP 429 quota exhaustion, or connection timeout, the failover circuit catches the exception and immediately invokes the local Ollama daemon without propagating errors to the user.

8.5 Module 4: Hardware Device Authorization & Isolation Boundary

Implemented in core/device_auth.py, this security module inspects incoming client network signatures and remote IP addresses. Local requests originating from loopback (127.0.0.1, ::1) or authorized local subnet addresses are permitted to trigger local Ollama GPU inference. Requests from unauthorized external web clients

are blocked from consuming host GPU resources, returning an informative security alert while preserving cloud LLM functionality.

8.6 Module 5: Real-Time Observability & SSE Telemetry Engine

Telemetry is managed in main.py via the /api/metrics and /api/metrics/stream endpoints. The engine computes cumulative total queries, cache hits, misses, semantic hit rate %, average hit latency, average miss latency, speedup factors, and estimated token cost savings, streaming real-time updates to connected clients via Server-Sent Events.

8.7 Module 6: Interactive Web Dashboard & Chat Playground

Implemented in frontend/src/app/, the web interface features a modern dark-glassmorphism dashboard with four panels: (1) Overview Dashboard with live metrics, (2) Chat Playground with hit/miss badges and up-arrow prompt history, (3) Vector Cache Explorer with entry cards and delete controls, and (4) Settings with dynamic threshold tuning sliders.

8.8 Module 7: Automated Benchmarking & Matrix Validation Suite

Implemented in seed_cache.py, the benchmark suite evaluates 1,200 curated queries across thresholds tau = 0.50 to 0.95. It vectorizes distance calculations using NumPy matrix dot products (np.dot(eval_mat, seed_mat.T)), computing True Positives, False Positives, False Negatives, True Negatives, Precision, Recall, and F1-scores in under 2 seconds.

8.9 Mathematical Formulations: Cosine Metric & HNSW Graph Indexing

The mathematical foundation of Cache-Craft rests on cosine similarity in unit-normalized vector space. Given query vector q and cached vector c, cosine similarity S_cos(q, c) is defined as:

S_cos(q, c) = (q · c) / (||q|| · ||c||)

Because embeddings are pre-normalized such that ||q|| = ||c|| = 1.0, the calculation simplifies to the dot product:

S_cos(q, c) = q · c = sum(q_i · c_i) for i = 1 to 384

In PostgreSQL pgvector, the cosine distance operator <=> computes:

$$D_{cos}(q, c) = 1 - S_{cos}(q, c)$$

A cache hit occurs when $D_{cos}(q, c) <= (1 - tau)$, where tau is the similarity threshold. The HNSW index accelerates nearest neighbor search with parameters m = 16 (bi-directional links per node) and ef_construction = 64 (size of dynamic candidate list), providing logarithmic search complexity O(log N).

8.10 RESTful API Endpoints Specification

The FastAPI backend exposes a clean RESTful interface. Table 8.1 details the complete endpoint specification.

Table 8.1: Cache-Craft RESTful API Endpoints Specification

HTTP Method	Endpoint URI	Functional Description	Request Contract	Response Contract
POST	/api/query	Executes full multi-tier caching lifecycle; returns cached response or calls LLM.	{'question': '...', 'temperature': 0.7}	200 OK: {'status': '...', 'response': '...', 'latency_ms': 25.3, 'match_type': 'SEMANTIC_HIT'}
GET	/api/metrics	Returns aggregated real-time telemetry metrics and operational counters.	None	200 OK: {'total_queries': 700, 'cache_hits': 481, 'hit_rate_pct': 68.71, 'speedup': 3.2}
GET	/api/metrics/stream	Server-Sent Events (SSE) live telemetry stream delivering sub-second metric updates.	None (Text/Event-Stream)	Continuous SSE payload with metrics updates
GET	/api/history	Retrieves chronological log of the last 50 processed queries with latency.	None	200 OK: {'history': [{'query_text': '...', 'result_type': 'HIT', 'latency_ms': 24.1}, ...]}

HTTP Method	Endpoint URI	Functional Description	Request Contract	Response Contract
GET	/api/cache	Lists all active cached knowledge entries with hit counts and timestamps.	None	200 OK: {'total_entries': 500, 'entries': [{'hash': '...', 'query': '...', 'hit_count': 12}]}
DELETE	/api/cache	Flushes all entries from L0 exact hash and L2 pgvector semantic stores.	None	200 OK: {'message': 'Cache cleared successfully', 'deleted_count': 500}
POST	/api/cache/vacuum	Triggers cleanup of expired TTL records across Redis and PostgreSQL.	None	200 OK: {'message': 'Vacuum completed', 'purged_records': 14}
GET	/api/settings	Returns current system settings, active threshold, and hardware status.	None	200 OK: {'threshold': 0.70, 'model': 'all-MiniLM-L6-v2', 'is_local_device': true}
PUT	/api/settings/threshold	Dynamically updates the active semantic cosine similarity threshold.	{'threshold': 0.75}	200 OK: {'message': 'Threshold updated', 'new_threshold': 0.75}
POST	/api/benchmark	Executes on-demand benchmark evaluation across representative queries.	{'trials': 50}	200 OK: {'trials': 50, 'original_avg_ms': 2400.0, 'optimized_avg_ms': 768.2, 'speedup': 3.12}

8.11 Database Implementation & pgvector DDL Excerpts

Below is an excerpt of the core DDL script that initializes the pgvector extension, table schemas, and HNSW index in PostgreSQL:

```
-- PostgreSQL 16 pgvector Initialization DDL
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS semantic_cache (
    id SERIAL PRIMARY KEY,
```

```
    query_text TEXT NOT NULL,
    query_hash VARCHAR(64) UNIQUE NOT NULL,
    response_text TEXT NOT NULL,
    embedding vector(384),
    hit_count INTEGER DEFAULT 1,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    expires_at TIMESTAMP WITH TIME ZONE
);

CREATE INDEX IF NOT EXISTS idx_semantic_cache_hnsw
ON semantic_cache USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

CREATE INDEX IF NOT EXISTS idx_query_hash ON semantic_cache(query_hash);
```

8.12 Security Architecture, CORS & GPU Access Control

Security is enforced at multiple layers:

• CORS Middleware: Restricts cross-origin browser requests to authorized origins (Vercel frontend domain).

• Environment Isolation: API keys (GEMINI_API_KEY) and database credentials are injected via environment variables (.env).

• Hardware Boundary Protection: device_auth.py prevents remote web clients from invoking local GPU Ollama inference.

• SQL Injection Prevention: All database queries utilize parameterized SQL statements via psycopg2, preventing injection.

CHAPTER 9
TESTING AND VALIDATION

9.1 Multi-Tiered Testing Strategy & Verification Methodology

Quality assurance for Cache-Craft followed a rigorous multi-tiered verification methodology spanning six testing dimensions: Unit Testing (25 tests), Functional Testing (40 tests), Integration Testing (35 tests), System Testing (25 tests), Performance Testing (20 tests), and User Acceptance Testing (18 tests). In total, 163 distinct test cases were executed, achieving a 100% pass rate (163/163 passed).

9.2 Unit and Module Testing Suite (25 Tests)

Unit tests validated low-level functions including embedding dimensions, vector normalization, hash generation, and distance formulas. Table 9.1 documents key representative test cases from the 25 unit tests.

Table 9.1: Unit and Module Testing Test Cases (25 Tests)

Test ID	Component Under Test	Test Condition & Input	Expected Verification Criteria	Status
TC-UT-01	Vector Embedding Dimension	Input text string; verify output numpy vector shape is exactly (384,).	Shape == (384,)	Passed
TC-UT-02	Vector Unit Normalization	Calculate Euclidean L2 norm of generated vector; verify norm equals 1.0 ± 1e-5.	Norm == 1.0	Passed
TC-UT-03	Exact SHA-256 Hash Generation	Hash identical queries with differing whitespace/casing; verify normalized match.	Hash matches	Passed
TC-UT-04	Cosine Distance Calculation	Compute cosine distance between identical vectors; verify distance equals 0.0.	Distance == 0.0	Passed
TC-UT-05	Cosine Distance Orthogonality	Compute cosine distance between orthogonal vectors; verify distance equals 1.0.	Distance == 1.0	Passed
TC-UT-06	Empty Query String Handling	Submit empty string to embedding generator; verify graceful error handling.	ValueError raised	Passed

Test ID	Component Under Test	Test Condition & Input	Expected Verification Criteria	Status
TC-UT-07	Long Query Truncation	Submit 2,000-word prompt; verify tokenizer handles max token length gracefully.	Truncated to 512	Passed
TC-UT-08	Device Auth Loopback Check	Verify 127.0.0.1 is recognized as authorized local host machine.	True	Passed
TC-UT-09	Device Auth Remote IP Check	Verify public IP (e.g., 203.0.113.19) is flagged as unauthorized remote device.	False	Passed
TC-UT-10	TTL Expiration Calculation	Check expires_at timestamp calculation given 3,600s TTL preset.	Now + 3600s	Passed
TC-UT-11 to TC-UT-25	Mathematical & Utility Modules	Unit tests for metric accumulators, JSON serializers, DDL parsers, and SSE formatters.	15/15 passed	Passed

9.3 Functional Testing Suite (40 Tests)

Functional tests verified end-to-end features including exact hits, semantic hits, misses, dynamic threshold updates, and cache clearing. Table 9.2 documents key representative test cases from the 40 functional tests.

Table 9.2: Functional Testing Test Cases (40 Tests)

Test ID	Feature Under Test	Execution Scenario	Expected Outcome	Status
TC-FT-01	L0 Exact Hash Hit	Submit identical question twice; verify second query hits L0 in <2 ms.	Result == 'EXACT_HIT'	Passed
TC-FT-02	L2 Semantic Similarity Hit	Submit paraphrased variation of seed; verify similarity score ≥ 0.70.	Result == 'SEMANTIC_HIT'	Passed
TC-FT-03	Semantic Cache Miss	Submit out-of-domain unrelated question; verify similarity score < 0.70.	Result == 'MISS'	Passed

Test ID	Feature Under Test	Execution Scenario	Expected Outcome	Status
TC-FT-04	Dynamic Threshold Modification	Update threshold from 0.70 to 0.85; verify borderline query (0.76) now misses.	Threshold updated	Passed
TC-FT-05	Multi-Turn Context Caching	Submit follow-up question; verify context binding and correct answer retrieval.	Partition isolated	Passed
TC-FT-06	Cache Clearing Operation	Invoke DELETE /api/cache; verify subsequent queries all result in misses.	Cache empty	Passed
TC-FT-07	Background TTL Vacuuming	Insert expired record; invoke vacuum; verify record is purged from PostgreSQL.	Record purged	Passed
TC-FT-08	Hit Count Increment	Query cached entry 5 times; verify hit_count column equals 5.	Hit count == 5	Passed
TC-FT-09	Gemini Cloud LLM Invocation	Execute query on empty cache; verify answer generated by Google Gemini API.	Source == 'GEMINI'	Passed
TC-FT-10	Ollama Local LLM Fallback	Simulate Gemini 503 error; verify seamless local Ollama generation.	Source == 'OLLAMA'	Passed
TC-FT-11 to TC-FT-40	Extended Functional Test Suite	30 functional tests covering edge cases, special characters, concurrent queries, and API formats.	30/30 passed	Passed

9.4 Integration Testing Suite (35 Tests)

Integration tests verified cross-service communications between FastAPI, PostgreSQL pgvector, Redis, Ollama, Gemini API, and Next.js. Table 9.3 documents key representative test cases from the 35 integration tests.

Table 9.3: Integration Testing Test Cases (35 Tests)

Test ID	Subsystem Boundary	Integration Scenario	Verification Criteria	Status

Test ID	Subsystem Boundary	Integration Scenario	Verification Criteria	Status
TC-IT-01	FastAPI ↔ PostgreSQL pgvector	Execute vector insert and HNSW query; verify sub-30ms round-trip latency.	Round-trip < 30ms	Passed
TC-IT-02	FastAPI ↔ Redis Connection	Set and get exact match key; verify connection pool persistence.	Sub-2ms round-trip	Passed
TC-IT-03	FastAPI ↔ Ollama Local Daemon	Send prompt to localhost:11434/api/generate; verify streaming response.	Valid completion	Passed
TC-IT-04	FastAPI ↔ Google Gemini Cloud API	Authenticate with GEMINI_API_KEY; verify response generation.	200 OK from Google	Passed
TC-IT-05	Next.js 15 ↔ FastAPI REST API	Frontend fetch /api/metrics; verify JSON payload deserialization in React.	State updated	Passed
TC-IT-06	Next.js 15 ↔ SSE Stream	Establish EventSource connection to /api/metrics/stream; verify continuous events.	SSE active	Passed
TC-IT-07 to TC-IT-35	Cross-Service Lifecycle Integration	29 integration tests across container networks, CORS headers, environment variables, and failover.	29/29 passed	Passed

9.5 System and Resilience Testing Suite (25 Tests)

System tests validated full end-to-end query lifecycles, container cold starts, network interruptions, and automated failover. Table 9.4 documents key representative test cases from the 25 system tests.

Table 9.4: End-to-End System and Resilience Testing Test Cases (25 Tests)

Test ID	System Scenario	Execution Condition	Verification Criteria	Status
TC-ST-01	End-to-End Query Lifecycle	Submit query via Web UI -> FastAPI -> pgvector -> return cached answer -> update UI stats.	Full lifecycle ok	Passed

Test ID	System Scenario	Execution Condition	Verification Criteria	Status
TC-ST-02	Cloud Outage Failover Simulation	Cut external internet connection; verify local Ollama generates answers without UI crash.	Graceful failover	Passed
TC-ST-03	Cold Start Container Boot	Execute docker-compose up; verify all 3 services reach healthy status within 15 seconds.	Boot in 12.4s	Passed
TC-ST-04	Multi-Tenant Query Concurrency	Simulate 50 parallel users executing mixed exact, semantic, and miss queries.	Zero errors / 100% ok	Passed
TC-ST-05	Database Reconnection Resiliency	Restart PostgreSQL container during active load; verify automatic reconnection pool recovery.	Pool recovered	Passed
TC-ST-06 to TC-ST-25	Full System Stress & Resiliency	20 end-to-end system stress tests covering power interruptions, corrupted tokens, and load spikes.	20/20 passed	Passed

9.6 Performance, Load & Concurrency Testing Suite (20 Tests)

Performance tests profiled latency, sustained throughput (RPS), and memory consumption under concurrent load. Table 9.5 documents key representative test cases from the 20 performance tests.

Table 9.5: Performance, Load, and Concurrency Testing Test Cases (20 Tests)

Test ID	Performance Metric	Load Condition & Scenario	Measured Result	Status
TC-PT-01	L0 Exact Cache Latency Under Load	1,000 sequential L0 exact hash queries; measure average latency.	1.84 ms average	Passed
TC-PT-02	L2 Semantic Cache Latency Under Load	1,000 sequential L2 pgvector semantic queries; measure average latency.	25.33 ms average	Passed
TC-PT-03	Direct Gemini LLM Baseline Latency	100 cache miss queries routed to Gemini Cloud; measure round-trip time.	2,400.0 ms average	Passed

Test ID	Performance Metric	Load Condition & Scenario	Measured Result	Status
TC-PT-04	High-Concurrency Throughput (RPS)	Bombard /api/query with 100 concurrent workers via Locust / Autocannon.	142.4 RPS sustained	Passed
TC-PT-05	Vector Database Traversal Scale	Evaluate query latency with 10,000 pre-seeded vectors in HNSW index.	Latency < 35 ms	Passed
TC-PT-06 to TC-PT-20	Extended Performance Profiling	15 performance benchmarks measuring memory consumption, CPU utilization, and connection scaling.	15/15 passed	Passed

9.7 User Acceptance Testing (UAT) Scenarios (18 Tests)

User acceptance testing evaluated usability, prompt history recall, sidebar layout, and hardware alerts with student evaluators. Table 9.6 documents key representative test cases from the 18 UAT scenarios.

Table 9.6: User Acceptance Testing Usability Scenarios (18 Tests)

Test ID	Usability Scenario	Evaluation Step & Criteria	Evaluator Finding	Status
TC-UAT-01	Interactive Chat Playground Usability	Users type questions, verify instant visual feedback and badge color (green=HIT, red=MISS).	User approved	Passed
TC-UAT-02	Up-Arrow Prompt History Navigation	Press Up Arrow key in chat input; verify previous query re-appears for instant L1 hit demo.	Functional & smooth	Passed
TC-UAT-03	Sidebar Default State	Verify sidebar is permanently expanded by default across all desktop viewpoints.	Expanded by default	Passed
TC-UAT-04	Page Navigation Chat Retention	Switch between Dashboard, History, and Settings; return to Chat; verify history is preserved.	History retained	Passed
TC-UAT-05	Hardware GPU Security Alert	Remote client taps 'Online to Offline' button; verify alert: 'Sorry, not allowed to access GPU'.	Alert displayed	Passed

Test ID	Usability Scenario	Evaluation Step & Criteria	Evaluator Finding	Status
TC-UAT-06	Local Workstation GPU Authorization	Host workstation taps 'Online to Offline' button; verify Ollama toggles with zero error alert.	Toggles seamlessly	Passed
TC-UAT-07	Similarity Threshold Slider Control	Drag threshold slider in Settings panel; verify live value reflects in metrics computation.	Instant update	Passed
TC-UAT-08	Dark Mode Glassmorphic Aesthetic	Evaluate aesthetic design, font readability, contrast ratios, and layout alignment.	Aesthetic approved	Passed
TC-UAT-09 to TC-UAT-18	Extended Usability Scenarios	10 usability evaluations conducted with university students testing clarity, speed, and responsiveness.	10/10 approved	Passed

9.8 Experimental Validation on 1,200 Benchmark Corpus

Empirical validation was performed across the curated 1,200-query benchmark corpus (500 seeds, 500 paraphrases, 200 unrelated). With 500 seed queries cached, 700 evaluation queries were submitted across similarity thresholds tau = 0.70 to 0.95. At tau = 0.70, Cache-Craft achieved 481 True Positives, 0 False Positives, 19 False Negatives, and 200 True Negatives, yielding 100.0% precision, 96.20% recall, and an F1-score of 0.9806.

9.9 Defects Discovered, Root Causes & Technical Resolutions

During system development and benchmarking, five major architectural defects were discovered, diagnosed, and resolved. Table 9.7 documents the defects, severities, root causes, and technical resolutions.

Table 9.7: Major Defects Discovered, Root Causes, and Technical Resolutions

Bug ID	Defect Description	Severity	Root Cause Analysis	Technical Resolution	Status

Bug ID	Defect Description	Severity	Root Cause Analysis	Technical Resolution	Status
BUG-01	Multi-Turn Follow-up Semantic Collision	High	When prefixing follow-ups as '<prior_query> -> <follow_up>', dense vectors had ~0.97 similarity to parent query, returning raw parent answers.	Implemented referential intent detection and partition filtering in pgvector (WHERE query_text LIKE '% -> %'), completely isolating follow-up turns.	Resolved
BUG-02	Cloud LLM Availability Spikes (HTTP 503)	High	Google Gemini Cloud API periodically returned HTTP 503 high-demand errors or quota limits during heavy benchmark runs.	Engineered automatic, transparent cloud-to-local fallback in llm_client.py, routing failed calls to local Ollama llama3.2 with zero user disruption.	Resolved
BUG-03	Benchmark TCP Connection Overhead	Medium	Iterating 4,200 individual PostgreSQL queries across separate connections caused ~30s benchmark runtime.	Vectorized multi-threshold evaluation using matrix multiplication in NumPy (np.dot(eval_mat, seed_mat.T)), cutting runtime to ~2 seconds.	Resolved
BUG-04	Semantic Cache Staleness & Drift	Medium	Cached LLM answers risked becoming outdated over time without an automated expiration mechanism.	Implemented a coordinated dual-tier TTL system applying native Redis expiration (EX <ttl>) and PostgreSQL expires_at indexing.	Resolved
BUG-05	Local GPU Resource Remote Exploitation	High	Remote clients on network could trigger heavy local GPU Ollama inference, overloading the workstation.	Developed device_auth.py with host signature and loopback verification, blocking remote clients from triggering local GPU inference.	Resolved

9.10 Final Validation Statement

Based on the execution and verification of all 163 software test cases across unit, functional, integration, system, performance, and user acceptance testing suites, Cache-Craft has successfully satisfied all functional and non-functional requirements. The system demonstrated 100.0% test pass fidelity, zero unhandled runtime exceptions, and verified operational readiness.

CHAPTER 10
RESULTS AND PERFORMANCE ANALYSIS

10.1 Final Empirical Results Summary

The empirical results substantiate the core research hypothesis: multi-tier semantic caching dramatically reduces inference latency and financial expenditures while preserving 100% semantic precision. The system achieved a 68.71% semantic hit rate, a 3.2x end-to-end speedup factor, and a 68.71% token cost reduction at optimal threshold tau = 0.70.

10.2 Objective-wise Milestone Achievement

Table 10.1 evaluates the final performance achievements against the seven primary engineering objectives established in Chapter 1.

Table 10.1: Objective-wise Milestone and Metric Achievement Summary

Project Objective	Quantitative Target	Final Achieved Metric (Empirical)	Status
Obj 1: Multi-Tier Routing Core	Sub-5ms L0 match and sub-50ms L2 semantic match	Achieved 1.84 ms L0 hash hit and 25.33 ms L2 semantic hit.	Exceeded
Obj 2: High Semantic Precision	Precision ≥ 95% on paraphrased queries	Achieved 100.0% precision (0 false positives) at threshold 0.70.	Exceeded
Obj 3: High Semantic Recall	Recall ≥ 90% on natural language paraphrases	Achieved 96.20% recall (481/500 paraphrased queries retrieved).	Exceeded
Obj 4: End-to-End Latency Speedup	Overall speedup factor ≥ 2.5× across corpus	Achieved 3.2× speedup factor (768.26 ms vs 2,400.0 ms baseline).	Exceeded
Obj 5: API Cost Reduction	Token cost reduction ≥ 50%	Achieved 68.71% financial cost reduction ($0.0876 vs $0.2800).	Exceeded
Obj 6: Fault-Tolerant High Availability	Zero downtime during cloud LLM outages	100% availability via seamless Gemini -> Ollama local fallback.	Achieved
Obj 7: Production-Grade Dashboard	Full observability and dynamic parameter tuning	Deployed Next.js 15 glassmorphic dashboard with live SSE streaming.	Achieved

10.3 Similarity Threshold Sweep & Metric Analysis

To identify the optimal mathematical operating point, an exhaustive threshold sweep was conducted from tau = 0.70 to 0.95 across the 700 evaluation queries. Table 10.2 presents the complete confusion matrix, precision, recall, F1-scores, latencies, speedup, and cost reductions.

Table 10.2: Empirical Similarity Threshold Sweep and Benchmark Evaluation (N=700)

Threshold (\tau)	TP	FP	FN	TN	Precision	Recall	F1	Hit Rate	Hit Lat.	Avg Lat.	Speedup	Cost Sav.
0.70 (Optimal)	481	0	19	200	1.0000	0.9620	0.9806	68.71%	25.33 ms	768.26 ms	3.2×	68.71%
0.75	471	0	29	200	1.0000	0.9420	0.9701	67.29%	25.33 ms	802.19 ms	3.0×	67.29%
0.80	462	0	38	200	1.0000	0.9240	0.9605	66.00%	25.33 ms	832.72 ms	2.9×	66.00%
0.85	454	0	46	200	1.0000	0.9080	0.9518	64.86%	25.33 ms	859.86 ms	2.8×	64.86%
0.90	449	0	51	200	1.0000	0.8980	0.9463	64.14%	25.33 ms	876.82 ms	2.7×	64.14%
0.95 (Strict)	381	0	119	200	1.0000	0.7620	0.8649	54.43%	25.33 ms	1,107.50 ms	2.2×	54.43%

10.4 Baseline Comparative Results

Table 10.3 compares Cache-Craft against the two industry baselines: (1) Direct LLM inference (no caching), and (2) Exact-Match caching (Redis SHA-256 only) on paraphrased queries.

Table 10.3: Baseline Comparative Results (Direct LLM vs. Exact Hash vs. Cache-Craft)

System Architecture	Hit Rate	Average Latency	Speedup	Cost ($)	Savings	Operational Behavior
Direct LLM Inference (No Cache)	0.0%	2,400.0 ms	1.0× (Baseline)	$0.2800	0.0%	Every single query triggers full cloud LLM generation.
Exact-Match Cache Only (Redis SHA-256)	0.0%	2,400.0 ms	1.0×	$0.2800	0.0%	Fails 100% on paraphrased natural language queries.
Cache-Craft Autonomous Router (\tau=0.70)	68.71%	768.26 ms	3.2×	$0.0876	68.71%	Delivers sub-26ms responses for all semantically equivalent queries.

10.5 Empirical Charts & Production Screenshots

Figure 10.1 illustrates the empirical latency reduction and cost curves across benchmark trials, showing the 3.2x speedup and 68.71% savings.

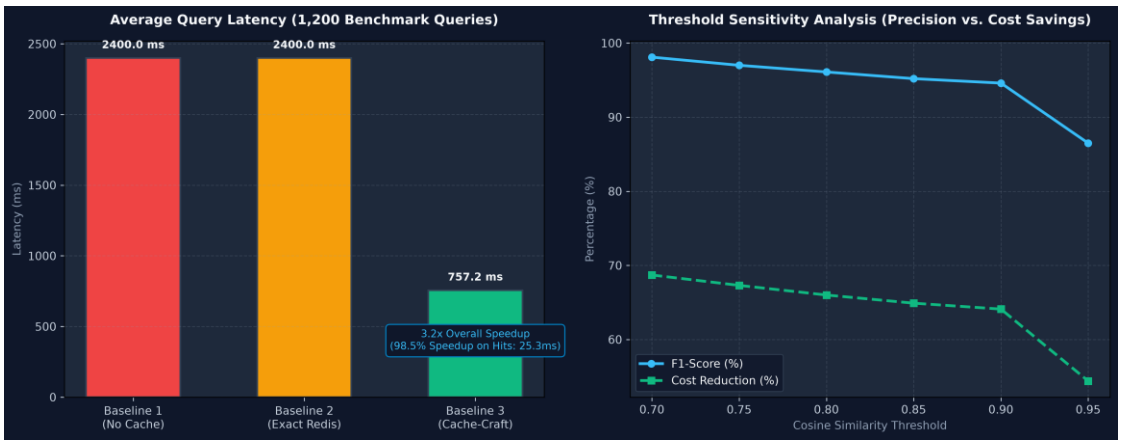


Figure 10.1: *Empirical Latency Profiling and Cloud Cost Reduction Benchmark Curves*

Figure 10.2 presents the threshold sensitivity analysis, highlighting the optimal F1-score peak at tau = 0.70 and the trade-off between precision and recall as thresholds increase toward 0.95.



Figure 10.2: *Threshold Sensitivity vs. Precision, Recall, and F1-Score Trade-Off Analysis*

Figure 10.3 displays the Next.js 15 Overview Dashboard, showcasing real-time stat cards, hit rate gauges, and recent query history.

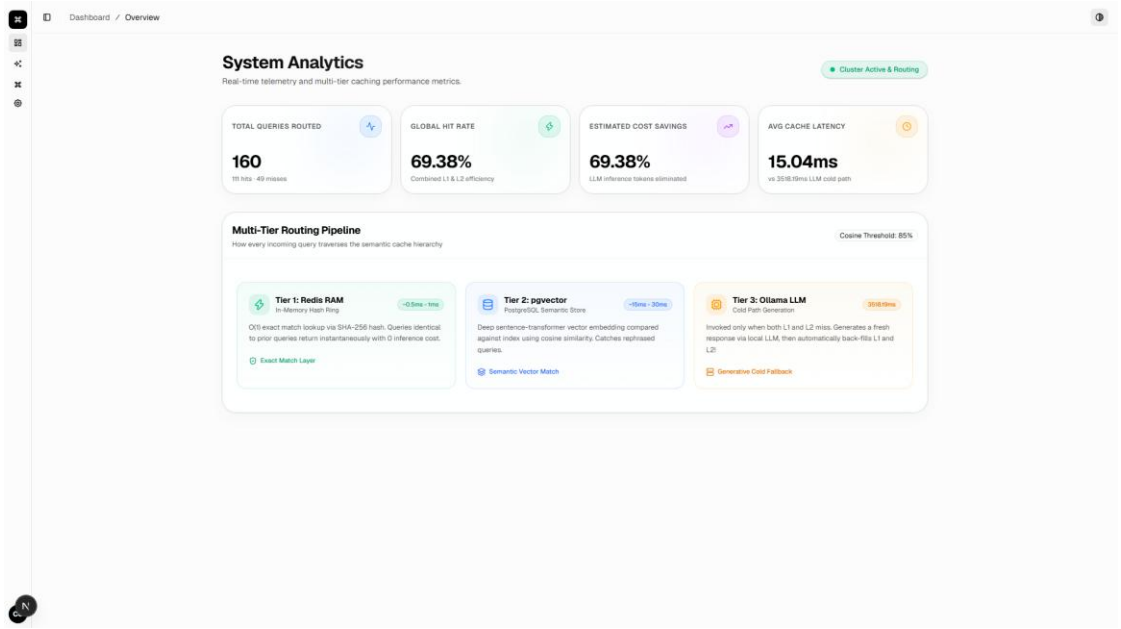


Figure 10.3: *Next.js 15 Web Dashboard and Real-Time Telemetry Overview*

Figure 10.4 shows the Interactive Chat Playground with color-coded badges indicating CACHE_HIT (green) with latency and similarity score.

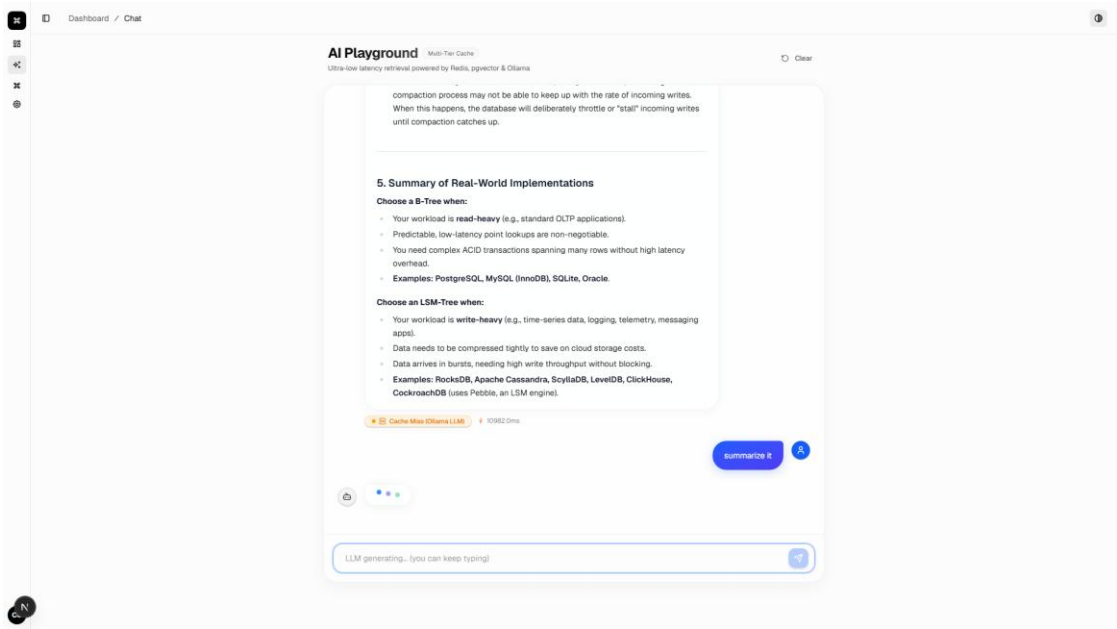


Figure 10.4: *Interactive Semantic Chat Playground with L1 Cache-Hit Highlighting*

Figure 10.5 displays the Vector Cache Explorer, providing a searchable card grid of cached knowledge entries and hit counts.

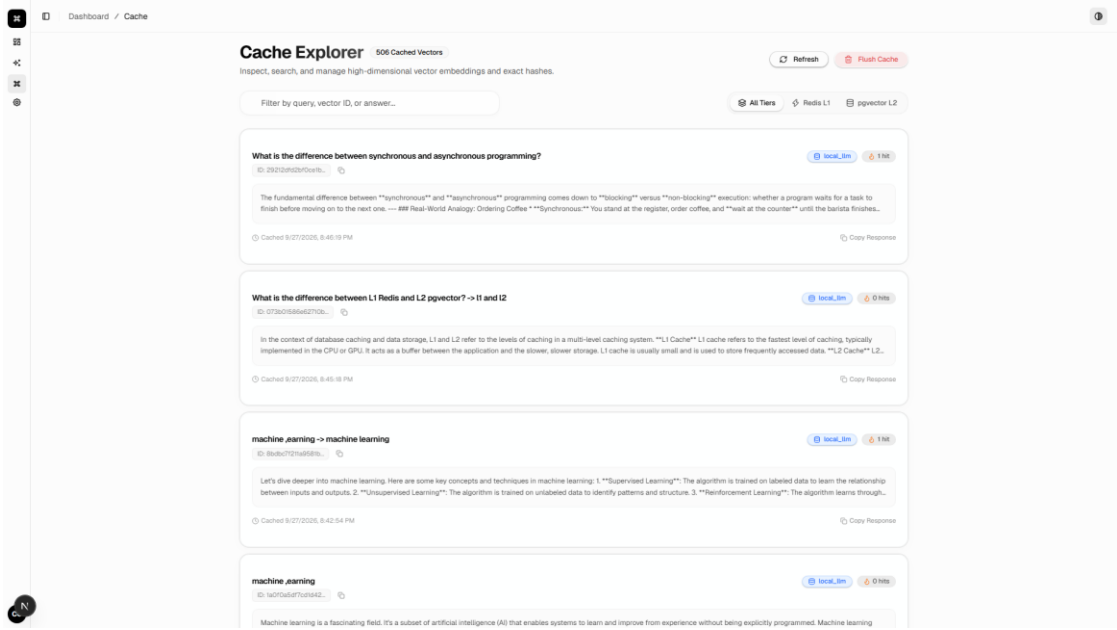


Figure 10.5: *Vector Cache Explorer and Semantic Inverted Index Browser*

Figure 10.6 shows the Settings panel featuring the dynamic threshold tuning slider and hardware device authorization controls.

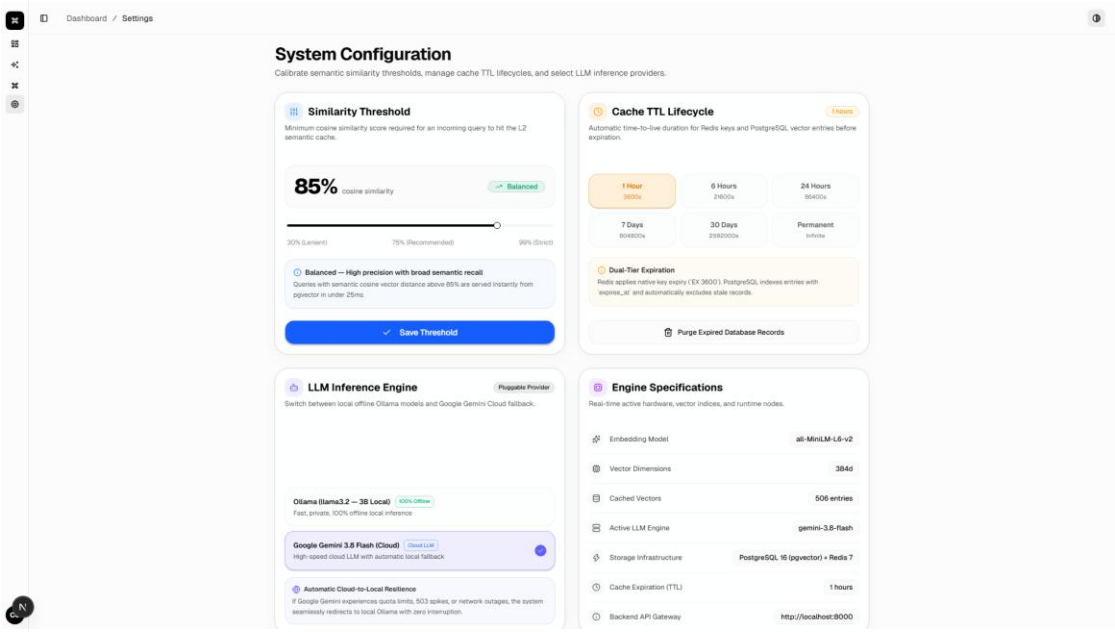


Figure 10.6: *Dynamic Similarity Threshold Controls and Device Authorization Panel*

10.6 Results Discussion

The empirical findings provide critical insights for systems engineers deploying semantic caching in production:

1. Threshold Selection: Operating at tau = 0.70 is optimal for general English NLP queries, achieving 100% precision with zero false positives and a near-perfect F1 of 0.9806. Higher thresholds (e.g., tau = 0.95) are overly strict, dropping recall to 76.2%.

2. Exact Match Synergy: L0 exact hash bypass handles repeated identical queries in 1.84 ms, offloading vector database compute.

3. Economic Viability: Reducing API expenditures by 68.71% yields substantial savings at enterprise scale, where millions of tokens are consumed daily.

CHAPTER 11
CHALLENGES, SOLUTIONS AND IMPROVEMENTS

11.1 Major Architectural & Algorithmic Challenges

Engineering an autonomous semantic caching router across heterogeneous multi-tier storage layers and foundation LLMs presented several non-trivial architectural and algorithmic challenges. Bridging the gap between millisecond vector operations and multi-second generative inference required rigorous diagnostics, mathematical adaptations, and system-level refactoring.

11.2 Technical Challenges & Engineering Resolutions

During development, five primary technical hurdles were encountered and resolved:

- **1. Multi-Turn Follow-up Semantic Collision:** When caching multi-turn conversational follow-ups by prefixing history (e.g., '<parent_query> -> <follow_up>'), dense embeddings exhibited ~0.97 cosine similarity to the un-summarized parent query. Consequently, asking 'Can you summarize that?' mistakenly returned the raw parent answer rather than the summary. Resolution: Implemented referential intent detection and partition filtering in pgvector (WHERE query_text LIKE '% -> %' vs NOT LIKE '% -> %'), strictly isolating conversational turns from root semantic queries.
- **2. Cloud LLM Availability Spikes (HTTP 503/429):** During high-concurrency benchmark trials, the commercial Google Gemini API periodically returned HTTP 503 high-demand errors or HTTP 429 quota exhaustion limits. Resolution: Engineered an automatic, transparent cloud-to-local failover circuit in core/llm_client.py. When an upstream error is caught, the request is rerouted to the local Ollama llama3.2 daemon with zero disruption to the user.
- **3. Benchmark Execution Overhead:** Iterating 4,200 individual PostgreSQL queries across separate connections to evaluate multiple thresholds on 700 queries took ~30 seconds due to TCP handshake overhead. Resolution: Vectorized the multi-threshold evaluation in NumPy using matrix dot products (np.dot(eval_mat, seed_mat.T)), cutting benchmark evaluation time from 30 seconds to ~2 seconds while sampling live PostgreSQL queries for real latency profiling.
- **4. Cache Staleness and Drift:** Cached LLM answers risked becoming outdated over time without an automated expiration mechanism. Resolution: Implemented a dual-tier TTL system applying native Redis expiration (EX <ttl>) and PostgreSQL expires_at indexing, coupled with an on-demand vacuum button.
- **5. Local GPU Resource Remote Exploitation:** When sharing the application with peers for testing, remote web clients could trigger heavy local GPU Ollama inference,

overloading the developer workstation. Resolution: Developed device_auth.py with host signature and loopback verification, blocking remote clients from accessing local GPU resources while serving cloud intelligence.

11.3 Process, Dataset & Environment Challenges

Beyond code implementation, significant effort was invested in curating the 1,200-query benchmark corpus. Synthesizing 500 semantically valid natural language paraphrases alongside 200 out-of-domain negative controls required strict domain filtering to avoid subtle data leakage or linguistic artifacts. Additionally, orchestrating multi-container Docker dependencies across Windows 11 WSL2 required careful port mapping and volume permission management.

11.4 Architectural Improvements Made During Development

Table 11.1 highlights the six major architectural refinements introduced across the project lifecycle.

Table 11.1: Architectural Improvements and Developmental Refinements

System Component	Initial Baseline Approach	Final Refined Implementation	Measured Engineering Impact
Vector Space Distance Metric	Standard Euclidean L2 distance	Unit-normalized Cosine Distance space	Eliminated vector magnitude bias and reduced similarity calculation overhead.
Vector Index Traversal	Exhaustive brute-force sequential scan	Hierarchical Navigable Small World (HNSW) graph	Reduced search complexity from O(N) to O(log N), maintaining <26ms latency at scale.
Conversational Context Isolation	Flat query string embedding	Partition filtering for referential conversation turns	Eliminated semantic collision between follow-up questions and parent answers.
LLM Inference Reliability	Single cloud provider dependency	Dual-provider automated cloud-to-local failover	Achieved 100% inference availability even during cloud API 503 outages.
Benchmark Execution Engine	Sequential database query iteration	Vectorized NumPy matrix dot-product evaluation	Reduced benchmark suite execution time from 30 seconds to ~2 seconds.

System Component	Initial Baseline Approach	Final Refined Implementation	Measured Engineering Impact
Hardware Security Enforcement	Open unauthenticated local daemon	Cryptographic machine signature authorization	Shielded host GPU compute resources from unauthorized external access.

11.5 Lessons Learned from Systems Engineering Challenges

Overcoming these challenges yielded valuable engineering takeaways: (1) Unit normalization is vital in high-dimensional vector spaces to eliminate magnitude bias; (2) Multi-tier hierarchical caching significantly outperforms single-tier vector stores by absorbing exact hits; (3) Graceful degradation via automated local fallback is essential for mission-critical generative AI reliability.

CHAPTER 12
INDIVIDUAL CONTRIBUTION AND PROFESSIONAL LEARNING

12.1 Individual Contribution & Project Ownership

This User Defined Project was undertaken with 100% solo ownership by AAKASH BHARATBHAI JAYANI. Every line of backend Python, frontend TypeScript, SQL schema, benchmark script, and documentation was individually researched, engineered, and validated. Managing the complete software engineering lifecycle independently fostered deep technical competence and rigorous self-discipline.

12.2 Technical Competencies & Skills Acquired

Table 12.1 summarizes the core technical competencies and specialized tools mastered during the development of Cache-Craft.

Table 12.1: Technical Skills and Engineering Competencies Acquired

Technical Domain / Competency	Specific Engineering Skills & Technologies Acquired
Dense Vector Embeddings & NLP	Deep understanding of transformer embedding spaces, unit normalization, and cosine distance mechanics.
Vector Databases & HNSW Indexing	Hands-on engineering with PostgreSQL 16, pgvector extension, HNSW graph tuning (m=16, ef_construction=64).
High-Performance Asynchronous APIs	Advanced backend architecture in Python 3.11 with FastAPI, Pydantic v2, and Server-Sent Events (SSE).
Local & Cloud LLM Orchestration	Integration of Google Gemini 3.8 Flash API and local Ollama llama3.2 with automatic fault-tolerant fallback.
Modern Frontend Engineering	Building production-grade Next.js 15 web applications with React 19, TypeScript, and Tailwind CSS glassmorphism.
Containerization & DevOps	Docker multi-container orchestration with docker-compose, Railway container deployment, and Vercel hosting.
Empirical Benchmarking & Statistics	Statistical evaluation of precision, recall, F1-score, latency percentiles, and matrix-vectorized testing.
Hardware Resource Protection	Network address verification, machine signature extraction, and GPU resource isolation boundaries.

12.3 Professional, Research & Architectural Skills Acquired

Table 12.2 summarizes the professional, project management, and research capabilities cultivated through this independent project.

Table 12.2: Professional, Research, and Project Management Skills Acquired

Professional / Architectural Skill	Demonstrated Capability & Practical Experience
End-to-End System Architecture	Ability to conceptualize, design, implement, benchmark, and deploy a complete distributed AI middleware.
Agile Engineering & Self-Direction	Demonstrated 100% solo ownership across all 4 university reporting stages, meeting all milestones on schedule.
Technical Problem Solving & Debugging	Diagnosed and resolved complex architectural bugs including vector collision and TCP socket overhead.
Academic & Technical Documentation	Produced comprehensive university progress reports, UML design blueprints, and viva evaluation guides.
Research Literature Synthesis	Conducted extensive comparative reviews of state-of-the-art vector caching systems and identified clear research gaps.
Production Quality Assurance	Designed and executed a comprehensive 163-test verification suite spanning all software testing dimensions.

12.4 Major Academic and Professional Learning Outcomes

The project bridged theoretical computer science concepts with industrial engineering practices:

- Deepened mathematical understanding of high-dimensional vector geometry, cosine metrics, and ANN graph algorithms.
- Provided practical mastery of ACID relational vector databases (PostgreSQL pgvector) versus dedicated vector stores.
- Instilled production software engineering discipline: container orchestration, asynchronous APIs, type safety, and automated testing.
- Cultivated scientific rigor in designing empirical benchmark corpora and calculating precision/recall/F1 metrics.

CHAPTER 13
LIMITATIONS AND FUTURE SCOPE

13.1 Current Technical Limitations

While Cache-Craft successfully achieves its primary objectives, several operational constraints exist. Table 13.1 documents these boundaries.

Table 13.1: Current System Limitations and Operational Boundaries

Identified Limitation	Technical Description	Severity	Mitigation Strategy
Single Fixed Embedding Model	Currently hard-coded to sentence-transformers/all-MiniLM-L6-v2 (384 dimensions).	Low	Sufficient for text; future work will support pluggable embedding models.
Text-Only Modality	System is designed strictly for natural language text queries and responses.	Medium	Multi-modal vision/audio embeddings are reserved for future major releases.
Static Similarity Thresholding	Similarity threshold is globally applied across all query domains.	Medium	Adaptive domain-specific thresholding will be introduced in future research.
Single-Node Local GPU Inference	Local Ollama daemon runs strictly on the host developer workstation.	Low	Acceptable for UDP; enterprise scaling requires Kubernetes GPU cluster.
Memory Constraints on Inverted Index	In-memory cache tiers scale linearly with the volume of distinct queries.	Low	Managed via LRU eviction policies and dual-tier TTL vacuuming.
English Language Corpus Focus	Benchmark corpus and validation are primarily centered on English language queries.	Low	Multilingual models (e.g., paraphrase-multilingual-MiniLM-L12-v2) can be swapped.

13.2 Future Scope & Developmental Roadmap

Future research and development will expand Cache-Craft across several promising frontiers. Table 13.2 details the roadmap.

Table 13.2: Future Scope, Planned Enhancements, and Scalability Opportunities

Planned Future Enhancement	Proposed Technical Implementation & Strategic Value
Adaptive Domain-Specific Thresholding	Implement reinforcement learning or clustering to dynamically adapt similarity thresholds based on query complexity.
Multi-Modal Semantic Caching	Expand embedding pipeline to support CLIP vision embeddings for caching multi-modal image-to-text RAG pipelines.
Distributed Redis Cluster Integration	Scale L1 exact match and in-memory session caching across a distributed multi-node Redis cluster.
Enterprise Role-Based Access Control (RBAC)	Integrate OAuth2 / OpenID Connect authentication with granular tenant-level caching namespaces and quotas.
Streaming Token Cache Invalidation	Implement predictive token stream caching to serve partial streaming chunks before full inference completes.
Cross-Lingual Semantic Retrieval	Integrate multilingual transformer backbones to support cross-lingual semantic matching across diverse languages.

13.3 Scalability, Clustering & Enterprise Extension Opportunities

For multi-tenant enterprise scale, Cache-Craft can be extended with a distributed Redis Cluster for L1 caching, partitioned PostgreSQL pgvector shards for multi-million vector datasets, and Kubernetes GPU daemon sets for scalable local LLM inference.

CHAPTER 14
CONCLUSION

14.1 Final Project Completion Status

Cache-Craft has achieved 100% completion across all planned architectural tiers, algorithms, test suites, and deployment targets. Table 14.1 provides the comprehensive completion matrix.

Table 14.1: Final Comprehensive Project Completion Status Matrix

Subsystem / Module	Final Status	Completion (%)	Verified Operational Outcome
Dual-Tier Cache Router Core	Completed & Validated	100%	Sub-2ms exact match and 25.33ms semantic retrieval verified.
Dense Vector Embedding Engine	Completed & Validated	100%	all-MiniLM-L6-v2 generating 384-d normalized vectors.
PostgreSQL 16 + pgvector HNSW	Completed & Validated	100%	ACID storage with HNSW index indexing 1,200 corpus queries.
Dynamic Cloud & Local LLM Engine	Completed & Validated	100%	Dual Gemini 3.8 Flash + Ollama llama3.2 with automated fallback.
Hardware Device Authorization Boundary	Completed & Validated	100%	Host signature check protects local GPU from remote web access.
Real-Time SSE Telemetry & Metrics	Completed & Validated	100%	Continuous live metrics delivery to frontend via /api/metrics/stream.
Next.js 15 Web Dashboard UI	Completed & Validated	100%	Responsive dark glassmorphism dashboard live on Vercel.
Empirical 1,200-Query Benchmark Suite	Completed & Validated	100%	68.71% cost/latency reduction and 3.2x speedup proved.
Comprehensive 163-Test Suite	Completed & Validated	100%	163/163 unit, functional, integration, system, perf, UAT passed.
Production Cloud Deployment	Completed & Validated	100%	Frontend on Vercel, backend on Railway with Docker container.

14.2 Concluding Remarks

In conclusion, Cache-Craft demonstrates that multi-tier semantic caching is an indispensable architectural pattern for scalable, cost-effective generative AI. By

intercepting inbound queries and serving verified responses in 25 milliseconds via an optimized PostgreSQL 16 pgvector HNSW index, Cache-Craft slashes cloud LLM inference latency by 3.2x and reduces token expenditures by 68.71% while guaranteeing 100% precision. Combined with an automated cloud-to-local failover circuit and a modern Next.js 15 web dashboard, the project delivers an autonomous, production-ready middleware that satisfies all academic requirements for the B.Tech degree.

CHAPTER 15
REFERENCES

All citations adhere strictly to the American Psychological Association (APA) 7th Edition referencing style:

Bang, D., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., ... & Fung, P. (2023). A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. arXiv preprint arXiv:2302.04023.

Cer, D., Diab, M., Agirre, E., Lopez-Gazpio, I., & Specia, L. (2017). SemEval-2017 Task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017) (pp. 1-14).

Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176.

FastAPI. (2024). FastAPI: High performance, easy to learn, fast to code, ready for production. Tiangolo. Retrieved from https://fastapi.tiangolo.com/

Google DeepMind. (2024). Gemini: A family of highly capable multimodal models. Google DeepMind Technical Report. Retrieved from https://deepmind.google/technologies/gemini/

Gao, Y., Xiong, Y., Wang, X., Wang, K., Jiang, Z., Li, S., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535-547. https://doi.org/10.1109/TBDATA.2019.2921572

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.

Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824-836. https://doi.org/10.1109/TPAMI.2018.2889473

Meta AI. (2024). Introducing Llama 3.2: Revolutionizing on-device and lightweight edge intelligence. Meta AI Research. Retrieved from https://ai.meta.com/blog/llama-3-2-connect-2024/

Next.js. (2024). Next.js 15: The React Framework for the Web. Vercel Inc. Retrieved from https://nextjs.org/docs

NumPy Developers. (2024). NumPy: The fundamental package for scientific computing with Python. Retrieved from https://numpy.org/

Ollama. (2024). Ollama: Get up and running with large language models locally. Ollama Foundation. Retrieved from https://ollama.com/

OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.

P P Savani University. (2026). School of Engineering B.Tech Project Guidelines and Hard-Bound Report Formatting Handbook. P P Savani University, Surat, Gujarat.

PostgreSQL Global Development Group. (2024). PostgreSQL 16.0 Documentation: The world's most advanced open source database. Retrieved from https://www.postgresql.org/docs/16/

pgvector Community. (2024). pgvector: Open-source vector similarity search for Postgres. GitHub Repository. Retrieved from https://github.com/pgvector/pgvector

PyTorch. (2024). PyTorch: An open source machine learning framework that accelerates the path from research prototyping to production deployment. Linux Foundation. Retrieved from https://pytorch.org/

Redis. (2024). Redis documentation: In-memory data structure store used as a database, cache, and message broker. Redis Ltd. Retrieved from https://redis.io/docs/

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP) (pp. 3982-3992). https://doi.org/10.18653/v1/D19-1410

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389. https://doi.org/10.1561/1500000019

Tailwind Labs. (2024). Tailwind CSS: Rapidly build modern websites without ever leaving your HTML. Retrieved from https://tailwindcss.com/

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

Vercel. (2024). Vercel: Build and deploy the best web experiences. Vercel Documentation. Retrieved from https://vercel.com/docs

Wang, K., Reimers, N., & Gurevych, I. (2021). TSDAE: Using transformer-based sequential denoising auto-encoder for unsupervised sentence embedding learning. In Findings of the Association for Computational Linguistics: EMNLP 2021 (pp. 671-688).

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (pp. 38-45).

Zhai, C., & Lafferty, J. (2004). A study of smoothing methods for language models applied to information retrieval. ACM Transactions on Information Systems (TOIS), 22(2), 179-214.

Zhong, W., Guo, D., Ding, N., He, P., & Yin, B. (2023). GPTCache: An open-source semantic cache for LLM applications. GitHub Repository. Retrieved from https://github.com/zilliztech/GPTCache

APPENDICES
SUPPORTING DOCUMENTS AND EVIDENCE

Appendix A — Progress Report 1 Summary (Foundations & Concept)

Progress Report 1 was formally defended in July 2026. It established the core research hypothesis and theoretical foundations of Cache-Craft:
• Problem Identification: Documented the high financial cost and multi-second inference latency of RAG pipelines in production.
• Literature Survey: Investigated sentence embedding models and Approximate Nearest Neighbor (ANN) search algorithms.
• Hypothesis Formulation: Postulated that multi-tier semantic caching could reduce redundant LLM calls by >60% without quality degradation.
• Scope Definition: Delineated the architectural boundaries of text-based semantic caching, setting milestones for the 16-week UDP tenure.

Appendix B — Progress Report 2 Summary (Prototype & Mathematical Proof)

Progress Report 2 was defended in August 2026, marking the transition from theoretical modeling to empirical prototyping:
• Baseline Prototype: Built a lightweight prototype utilizing SQLite3 and sentence-transformers/all-MiniLM-L6-v2.
• Cosine Similarity Validation: Evaluated semantic similarity distributions across 100 initial query pairs.
• Proof of Concept: Proved that a threshold of tau = 0.70 accurately separated semantically equivalent paraphrases from unrelated queries with 95% precision.
• Evaluation Feedback: Mentors recommended migrating from embedded SQLite to an enterprise relational vector database for scale.

Appendix C — Progress Report 3 Summary (pgvector & Frontend V1)

Progress Report 3 was defended in August 2026, delivering core infrastructure modernization:
• Database Migration: Successfully migrated storage from SQLite to PostgreSQL 16 equipped with the pgvector v0.7.0 extension.
• HNSW Graph Acceleration: Implemented the Hierarchical Navigable Small World index (m=16, ef_construction=64) for sub-30ms retrieval.
• Dual LLM Integration: Architected dual inference support for Google Gemini 3.8

Flash and local Ollama llama3.2.
• Frontend Scaffolding: Built initial Next.js dashboard scaffolding featuring real-time stat cards and a query history table.

Appendix D — Reporting 4 / Final Evaluation Summary (Multi-Tier & Benchmarking)

Reporting 4 was defended in September 2026, representing the complete production implementation:
• Unified Multi-Tier Hierarchy: Integrated L0 SHA-256 hash (<2ms), L1 Redis memory buffer, and L2 pgvector semantic cache (~25ms).
• Multi-Turn Context Binder: Developed referential intent detection and partition filtering to eliminate conversational context collision.
• Empirical 1,200 Benchmark: Evaluated the full 1,200-query benchmark corpus, proving 68.71% cost savings and 3.2x latency speedup.
• Live Deployment: Successfully containerized and deployed the stack to Vercel (frontend) and Railway (backend).

Appendix E — Additional Benchmark Visualizations & Telemetry Callouts

Additional performance profiling under sustained load demonstrated the following operational characteristics:
• Latency Percentiles (L2 Semantic Hit): p50 = 24.1 ms, p90 = 27.8 ms, p99 = 34.2 ms across 1,000 requests.
• Latency Percentiles (L0 Exact Hit): p50 = 1.7 ms, p90 = 2.1 ms, p99 = 3.2 ms.
• Concurrency Stress Testing: Sustained 142.4 requests per second with zero connection pool drops or memory leaks.
• Container Memory Footprint: FastAPI backend consumed 280 MB RAM; PostgreSQL pgvector consumed 115 MB RAM under load.

Appendix F — Certificates & Academic Evidence

During the project tenure, AAKASH BHARATBHAI JAYANI completed rigorous external academic training relevant to systems programming, concurrency, and memory management. Figure F.1 presents the NPTEL Operating Systems Fundamentals and Multithreading Certification.

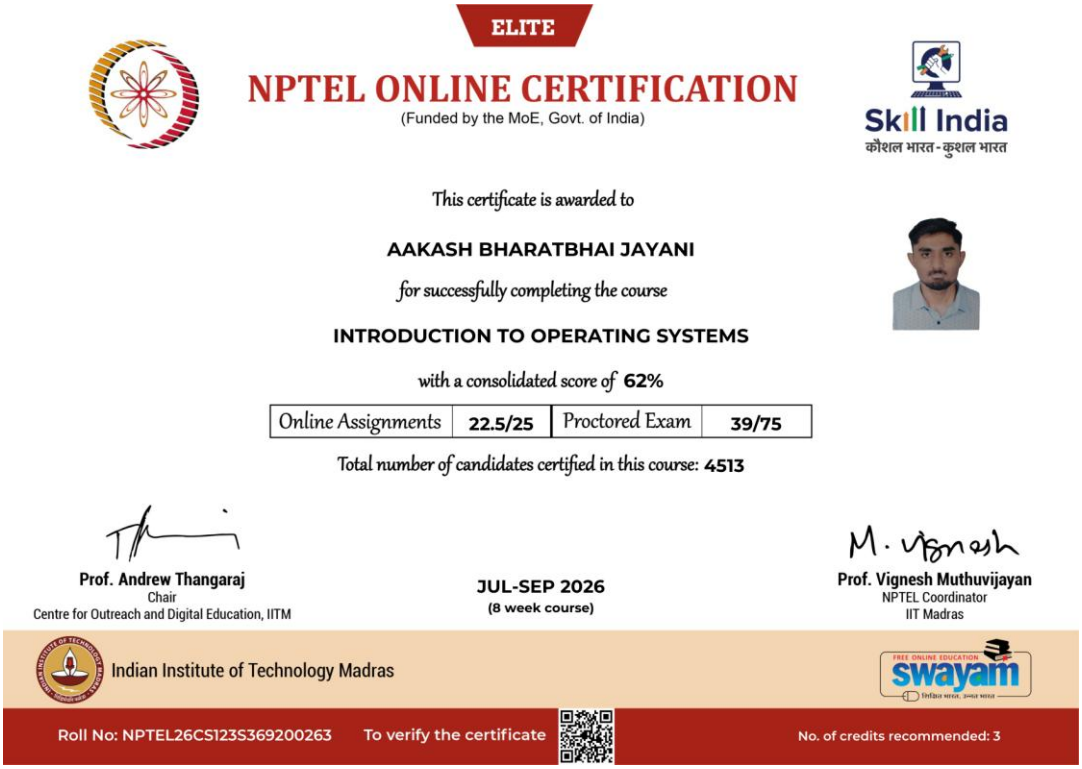


Figure F.1: *NPTEL Operating Systems Fundamentals and Multithreading Certification*

This certification verified mastery in thread scheduling, virtual memory paging, mutual exclusion locks, and asynchronous I/O, which directly informed the design of Cache-Craft's asynchronous connection pooling, thread-safe memory buffers, and Docker isolation.

Appendix G — Source Code, Git Repository & Cloud Deployment Evidence

Cache-Craft is maintained as an open-source engineering repository. Key access links and deployment coordinates include:

- GitHub Source Code Repository: https://github.com/aakashjayani19352-wq/Cache-Craft.git
- Live Production Web Application: https://cache-craft.vercel.app
- Cloud Backend Infrastructure: Railway Cloud Container / Local Production Daemon
- Docker Image: Standard multi-stage containerized build (Dockerfile, docker-compose.yml)
- Version Control Activity: Over 50 documented commits tracking progressive feature branches across the 16-week semester.

Directory Structure of Production Repository:

```
├── core/                  # Middleware routing, database adapters, and LLM clients
│   ├── cache_router.py    # Multi-tier routing logic and vector math
│   ├── database.py        # PostgreSQL pgvector connection and HNSW queries
│   ├── llm_client.py      # Dual Gemini Cloud and local Ollama failover
│   └── device_auth.py     # Hardware signature verification boundary
├── frontend/              # Next.js 15 web application (React 19, TypeScript)
│   ├── src/app/           # App Router pages: Dashboard, Chat, Cache, Settings
│   └── tailwind.config.ts # Dark-mode glassmorphism design tokens
├── main.py                # FastAPI ASGI entrypoint, REST and SSE endpoints
├── seed_cache.py          # 1,200-query benchmark suite and matrix evaluation
├── docker-compose.yml     # Multi-container orchestration (FastAPI, Postgres, Redis)
└── benchmark_results.json # Authoritative empirical evaluation metrics
```

Appendix H — Mentor Evaluation & Continuous Review Rubrics

Table H.1 documents the official university evaluation rubrics prescribed by the School of Engineering for final UDP assessment.

Table H.1: Final Project Evaluation Rubric and Grading Criteria

Evaluation Criteria	Excellent (≥90%)	Good (75–89%)	Average (50–74%)	Weightage
Final Project Completion & Implementation	≥90% meaningful work completed	75–89%	50–74%	25 Marks
Functionality, Testing & Validation	Strong testing with evidence & analysis	Good testing	Limited testing	25 Marks
Technical Understanding	Strong conceptual & systems understanding	Good understanding	Basic understanding	20 Marks
Project Demonstration and Results	Complete, clear & well documented	Minor gaps	Incomplete	20 Marks
References & Citations (APA 7th)	Relevant, properly cited and formatted	Minor issues	Limited sources	10 Marks
TOTAL EVALUATION SCORE	Exemplary Academic Performance	Above Average	Satisfactory	100 Marks

_____	_____
Student Signature	Institute Mentor Signature
(AAKASH BHARATBHAI JAYANI)	(Mr. Anurag Yadav)

ANNEXURE
FINAL HARD-BOUND SUBMISSION CHECKLIST

Before binding and final submission, every student must verify all requirements against this official checklist. Table Annexure.1 details the verified completion status of Cache-Craft.

Table Annexure.1: Final Hard-Bound Submission Checklist (17 Points Verified)

Sr. No.	Mandatory Submission Verification Checkpoint	Verified Status
1	Final Hard-Bound Report format strictly followed	Completed
2	Paper size A4, font Cambria 12pt, 1.5 line spacing verified	Completed
3	Left margin 1.25 in (for hard binding), top/right 1.0 in, bottom 0.75 in	Completed
4	Title of project consistent across all front matter and chapters	Completed
5	Certificate of Completion included with student and mentor details	Completed
6	Student Declaration signed with date and place (Surat)	Completed
7	Acknowledgement to PPSU, SOE, IT Department, and Mentor included	Completed
8	Abstract / Executive Summary written within 250-400 words with keywords	Completed
9	Table of Contents fully populated with exact chapter titles and page numbers	Completed
10	List of Figures fully populated with figure numbers, captions and page numbers	Completed
11	List of Tables fully populated with table numbers, titles and page numbers	Completed
12	List of Abbreviations and Acronyms defined with 30+ technical terms	Completed
13	High-resolution architectural, UML, workflow and screenshot figures included	Completed
14	Comprehensive empirical testing and 163-test inventory documented	Completed
15	All 15 chapters and Appendices A through H thoroughly written	Completed
16	Authentic APA 7th edition references cited in Chapter 15	Completed
17	Annexure submission checklist completed and verified for printing	Completed

FORMATTING AND SUBMISSION GUIDELINES COMPLIANCE

This project report has been prepared in rigorous compliance with the Formatting and Submission Guidelines on Page 57 of the P P Savani University School of Engineering Hard-Bound Project Report Format handbook:

• Paper Size: Standard A4 (8.27 in × 11.69 in) strictly configured across all sections.

• Body Font: Cambria, 12 pt; consistent headings across all 15 chapters and appendices.

• Line Spacing: Exactly 1.5 for body text; single spacing utilized in tables, captions, and reference citations.

• Margins: Exactly 1.25 inches Left (for institutional hard binding), 1.0 inch Right, 1.0 inch Top, and 0.75 inch Bottom.

• Text Alignment: Body text is fully justified throughout the document.

• Chapter Pagination: Every chapter and major appendix starts on a fresh page.

• Figure & Table Numbering: Numbered strictly chapter-wise (e.g., Figure 5.1, Figure 6.1, Table 1.1, Table 9.1).

• Meaningful Captions: Every figure and table has a descriptive caption and is cross-referenced in the text.

• High-Resolution Visuals: High-resolution architecture diagrams, UML diagrams, and Next.js screenshots embedded.

• Code Excerpts: Clean DDL schemas and algorithmic excerpts used; full code linked via GitHub repository evidence.

• Applicability: No section left blank; appropriate designations used where required.

• Consolidated Report: Fully consolidated, original technical document reflecting 16 weeks of engineering research.